HireMe | AI

# AI Resume Screening Platform

Project documentation, system design
and working guide for candidates and recruiters

**HireMe AI**

Version 1.0 | 4 November 2026

Live website: https://hireme-ai-hiring.vercel.app

All names, email addresses and phone numbers shown in the screenshots are sample data.

# Contents

**1 Introduction . . . 4**

1.1 Who uses it . . . 4

1.2 The platform at a glance . . . 4

**2 Technology stack . . . 5**

**3 System architecture . . . 6**

3.1 Where each part runs . . . 7

3.2 How the pieces stay safe . . . 7

**4 Backend architecture . . . 8**

4.1 The layers, in plain words . . . 9

4.2 A request from start to finish: "Apply to a job" . . . 9

**5 Database design . . . 10**

5.1 What each table stores . . . 12

5.2 How the database is changed safely . . . 12

**6 The AI service and the match score . . . 13**

6.1 The formula . . . 13

6.2 Endpoints of the AI service (private, API key required) . . . 13

**7 Security, privacy and rules . . . 14**

7.1 The four job rules a recruiter can choose . . . 14

**8 Files and folders . . . 15**

8.1 The folder tree . . . 15

8.2 Frontend routes . . . 16

8.3 Count the files today . . . 16

**9 Flows with arrows . . . 17**

9.1 Sign-up and email verification . . . 17

9.2 The one-time code, used four ways . . . 18

9.3 Upload and analyse a resume . . . 18

9.4 Apply to a job . . . 19

9.5 The hiring pipeline . . . 20

**10 Working guide: pages everybody sees . . . 21**

Landing page: first screen . . . 21

Landing page: how it works . . . 22

Landing page: how the match is explained . . . 23

Landing page: two sides of hiring . . . 24

Create account (top of the form) . . . 25

Create account (bottom) and footer . . . 26
Log in . . . 27
**11 Other pages for visitors . . . 28**
**12 Working guide: the candidate . . . 29**
Candidate: Dashboard (top) . . . 29
Candidate: Dashboard (lower part) . . . 30
Candidate: My resumes . . . 31
Candidate: Browse jobs . . . 32
Candidate: Profile page . . . 33
Candidate: change password and delete account . . . 34
12.1 Everything a candidate can do . . . 35
**13 Working guide: the recruiter . . . 36**
Recruiter: Dashboard (top) . . . 36
Recruiter: Dashboard (lower part) . . . 37
Recruiter: My jobs . . . 38
Recruiter: Analytics (summary) . . . 39
Recruiter: Analytics (charts) . . . 40
Recruiter: Activity feed . . . 41
Recruiter: Profile page . . . 42
Recruiter: delete account . . . 43
13.1 Posting a job and choosing the rules . . . 44
13.2 Everything a recruiter can do . . . 44
**14 API reference . . . 45**
**15 Emails . . . 46**
**16 Testing, deployment and settings . . . 47**
16.1 Automatic tests . . . 47
16.2 Settings the system needs (names only, no values) . . . 48
16.3 Running it on a laptop . . . 48
**17 Glossary . . . 49**

# 1 Introduction

The **AI Resume Screening Platform** is a website where candidates upload resumes and apply to jobs, and recruiters post jobs and receive the applicants already ranked by an AI match score. The AI reads each resume, finds the skills, compares them with the skills the job asks for, and gives a score from 0 to 100. People always make the final decision: the score only supports it.

This document explains the whole system from A to Z: how it is built, where every part runs, how the data is stored, how each flow works step by step, what every file and API is for, and how a candidate and a recruiter use each screen. Screenshots carry numbered message boxes that explain each button.

## 1.1 Who uses it

| Person | What they do | Main screens |
|---|---|---|
| **Candidate** | Uploads resumes, browses jobs, accepts the job rules, applies, follows the status and interview details. | Dashboard, My Resumes, Browse Jobs, My Activity, Profile |
| **Recruiter** | Posts jobs with rules, reviews ranked applicants, shortlists, rejects, hires, schedules interviews, reads analytics. | Dashboard, My Jobs, Analytics, Activity Feed, Profile |
| **Visitor** | Reads the landing page, the Privacy Policy and the Terms of Service, then creates an account or logs in. | Landing, Create account, Log in, legal pages |

## 1.2 The platform at a glance

| Item | Value |
|---|---|
| **Live website** | https://hireme-ai-hiring.vercel.app |
| **Roles** | 2: Candidate and Recruiter (login with email and password, or Google) |
| **Hiring stages** | 6: Applied, Shortlisted, Scheduled, Completed, Hired, Rejected |
| **Database** | Microsoft SQL Server (Azure SQL in production), about 16 tables |
| **Backend** | ASP.NET Core Web API (.NET 10) with Entity Framework Core |
| **Frontend** | React single-page app built with Vite |
| **AI service** | Python FastAPI service: reads PDF and DOCX, MiniLM embeddings, Groq language model |
| **Automatic tests** | 252 in total: 121 backend (xUnit) and 131 frontend (Vitest) |
| **Emails** | 12 kinds, all sent as branded HTML with a plain-text fallback |

**How to read this document.** Sections 2 to 7 are the technical design. Section 8 is the file map. Section 9 follows each important flow with arrows. Sections 10 to 13 are the working guide: one part for everybody who is not logged in, one for the candidate and one for the recruiter. Sections 14 to 17 cover the API, emails, testing and deployment, and a glossary.

# 2 Technology stack

| Layer | Technology | What it is used for |
|---|---|---|
| **Frontend** | React, Vite, React Router, Axios, Tailwind CSS plus a custom style sheet, Recharts, docx-preview, @react-oauth/google | All pages, routing between them, calls to the API, charts, preview of DOCX resumes, Google sign-in button |
| **Frontend tests** | Vitest, Testing Library, jsdom | Automatic tests of pages and helpers; they run on every push |
| **Backend** | ASP.NET Core Web API (.NET 10), Entity Framework Core, JWT bearer authentication, BCrypt, MailKit, Google.Apis.Auth | Business rules, login tokens, password hashing, emails, database access |
| **Backend tests** | xUnit, Moq | Tests of services, rules, emails and validation |
| **AI service** | Python 3.12, FastAPI, Uvicorn, sentence-transformers (all-MiniLM-L6-v2), pdfplumber, python-docx | Reads resume text, compares skills by meaning, scores the match |
| **Language model** | Groq API (default model openai/gpt-oss-20b) | Turns resume or job text into a clean list of skills (JSON) |
| **Database** | Microsoft SQL Server; Azure SQL in production, a Docker container on the developer laptop | All permanent data |
| **Email** | Brevo SMTP | Sends every email (codes, welcome, status changes) |
| **Security services** | Cloudflare Turnstile, Google Sign-In | Human check on forms; optional login with Google |
| **Hosting** | Vercel (website), Azure App Service (API, East Asia), Azure container (AI service, Korea Central), Azure SQL (East Asia) | Where each part runs |
| **Delivery** | GitHub, GitHub Actions, Vercel previews | Source control, automatic tests and automatic deployment |

# 3 System architecture

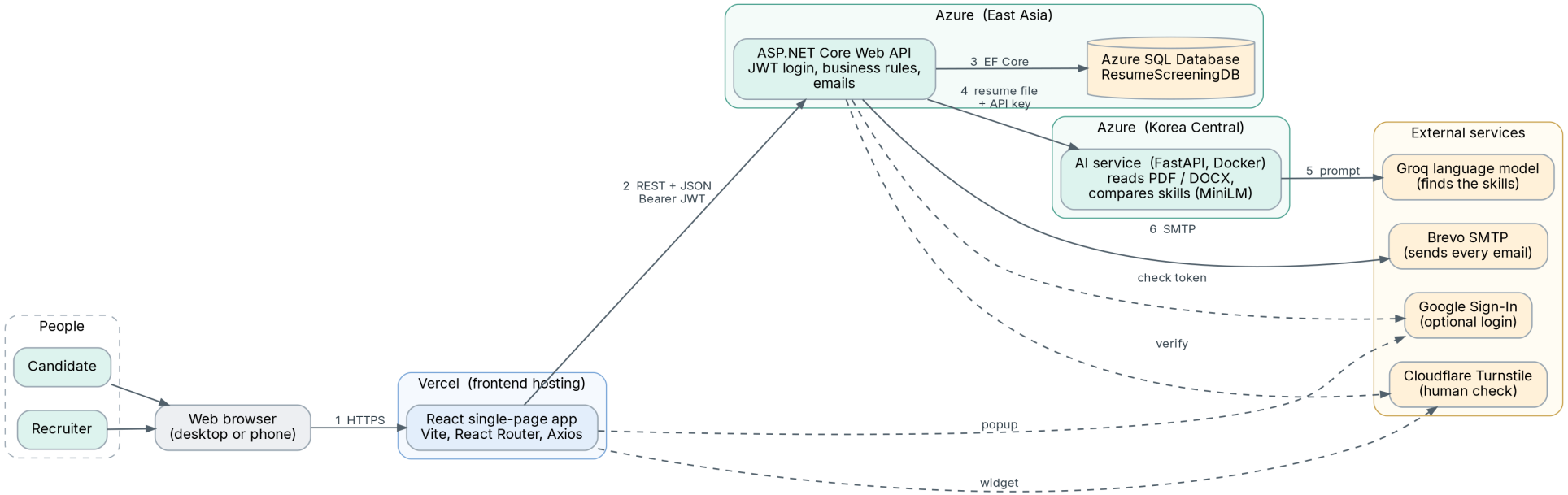



Figure 1. Everything that runs when somebody uses the website. Solid arrows are the normal path of a request, dashed arrows are optional or checking steps.

| # | What happens | Details |
|---|---|---|
| 1 | The browser loads the website from **Vercel** | A React single-page app. After the first load, pages change without reloading. |
| 2 | The app calls the **API** with REST and JSON | Every call after login carries the login token (JWT) in the Authorization header. A 401 answer logs the person out. |
| 3 | The API reads and writes **Azure SQL** | Entity Framework Core. Migrations run by themselves when the API starts. |
| 4 | The API sends the resume file to the **AI service** | A private service protected by a shared API key. It runs in Korea Central because that region had capacity. |
| 5 | The AI service asks the **Groq language model** for the skills | Only text goes out: the resume text or the job text. |
| 6 | The API sends emails through **Brevo** | Codes, welcome, application and status emails, in HTML and plain text. |

## 3.1 Where each part runs

| Part | Host | Region | Updates |
|---|---|---|---|
| **React website** | Vercel | Global network | Automatic on every merge to main; a preview for every pull request |
| **ASP.NET Core API** | Azure App Service (the resume API app) | East Asia | GitHub Action "Deploy backend" on every merge to main |
| **SQL database ResumeScreeningDB** | Azure SQL | East Asia | EF Core migrations at API start; some changes were also applied by hand with SQL |
| **AI service** | Azure container built from AiService/Dockerfile | Korea Central | Rebuilt and pushed as a Docker image |
| **Emails** | Brevo (SMTP) | Cloud | No deployment: only settings |
| **Human check** | Cloudflare Turnstile | Cloud | Site key in Vercel, secret key in an Azure app setting |

## 3.2 How the pieces stay safe

- **Login token:** after login the API returns a JWT valid for 8 hours. The browser keeps it in local storage and sends it with every request. When it expires the person is sent back to the login page.
- **Roles:** every protected endpoint checks the token. A person can only read or change their own data (the API answers 403 for any other id).
- **Secrets** (database password, Groq key, Turnstile secret, AI service key) are never in the code: they are app settings on Azure or Vercel.
- **CORS:** the API accepts browser calls only from the website address.

# 4 Backend architecture

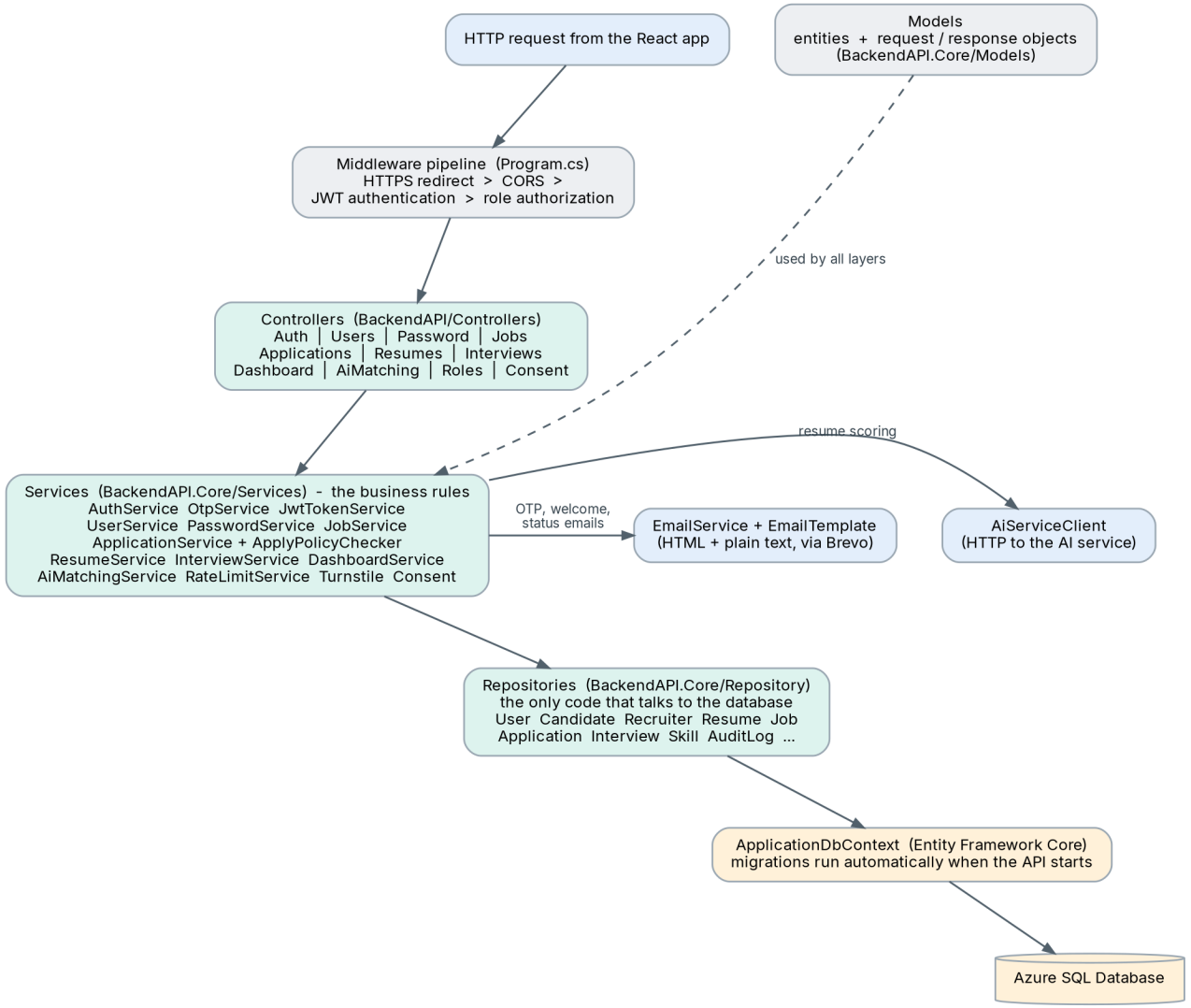



Figure 2. The layers of the API. Each layer only talks to the one below it.

## 4.1 The layers, in plain words

| Layer | Folder | Job | Example |
|---|---|---|---|
| **Middleware** | BackendAPI/Program.cs | Starts the server, registers every service, sets HTTPS, CORS, JWT login and role checks, and runs the database migrations at start. | A request without a valid token never reaches a controller. |
| **Controllers** | BackendAPI/Controllers | Receive the HTTP request, check the input, call one service and return the answer with the right status code (200, 400, 401, 403, 404). | ApplicationsController.Apply |
| **Services** | BackendAPI.Core/Services | All the business rules: who may do what, what must be checked, which email goes out, how the AI is called. | ApplicationService + ApplyPolicyChecker enforce the job rules |
| **Repositories** | BackendAPI.Core/Repository | The only code that talks to the database. Each one hides the queries of one table. | ApplicationRepository |
| **Models** | BackendAPI.Core/Models | Table classes (entities) and the request and response objects the API sends and receives. | Application.cs, ApplicationRequest.cs |
| **Migrations** | BackendAPI.Core/Migrations | Step-by-step changes of the database structure, in order. | AddJobPolicies adds the job rule columns |
| **Tests** | BackendAPI.Tests | 107 automatic tests of services, rules, emails and validation. | ApplyPolicyChecker tests |

## 4.2 A request from start to finish: "Apply to a job"

- The candidate presses **Apply**. The React page sends POST /api/Applications with the job id, the resume id and acceptedPolicies = true, plus the login token.
- Middleware checks the token and the role. The **controller** checks that acceptedPolicies is true and passes the request to **ApplicationService**.
- **ApplyPolicyChecker** reads the job rules and the history of the candidate: one job at a time, a pause after a hire, a wait after a rejection. If a rule blocks the application the answer is 400 with a message in plain words.
- The **repository** saves the application with status Applied and the time the rules were accepted.
- **AiMatchingService** sends the resume file and the job text to the AI service and stores the final score on the application.
- **EmailService** sends two emails: "Application submitted" to the candidate and "New applicant" to the recruiter.
- The answer returns to the page, which shows the match ring and the status.

# 5 Database design

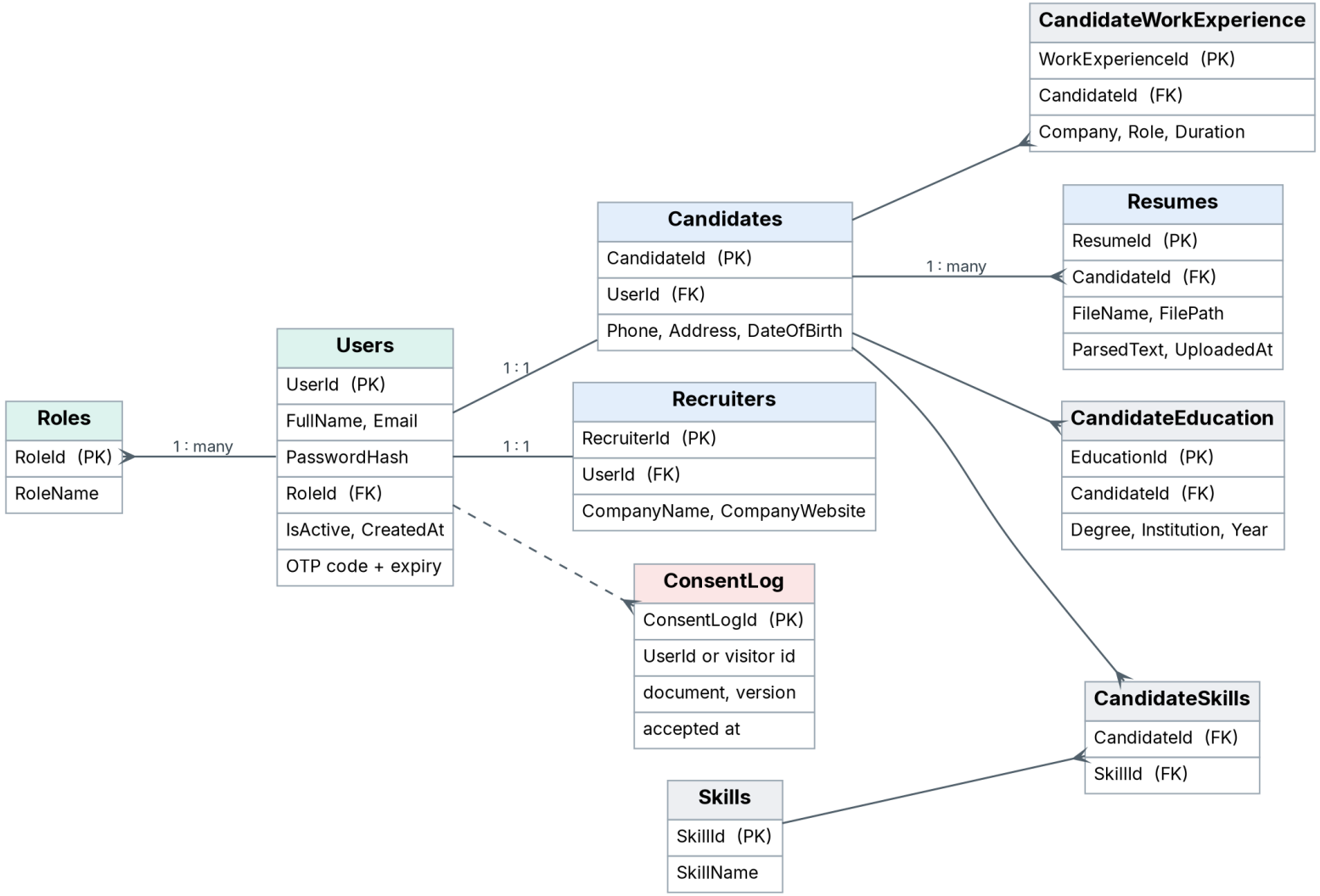



Figure 3. People, resumes and skills. PK = primary key, FK = foreign key. A crow's foot means "many".

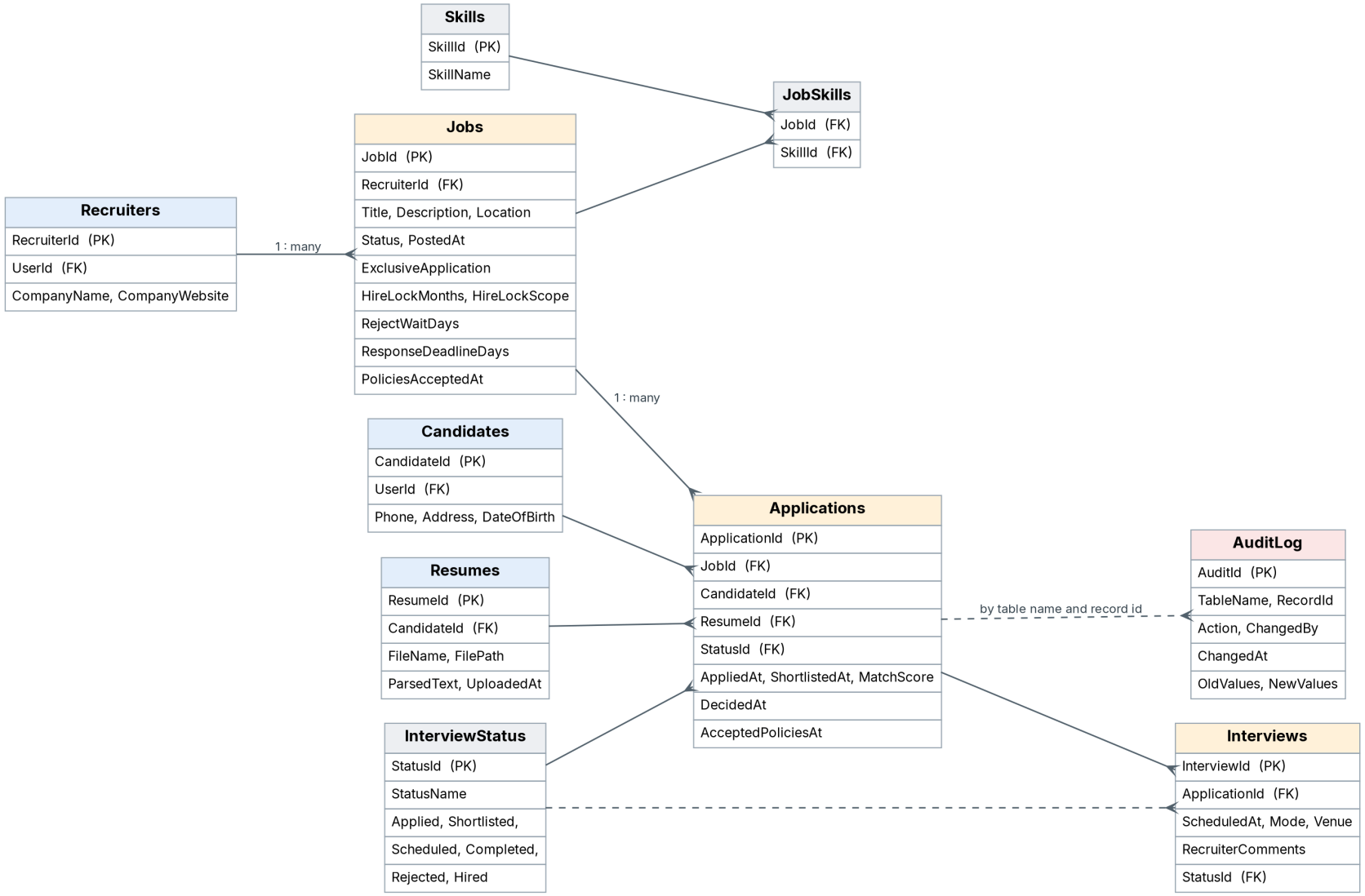



Figure 4. Jobs, applications, interviews and the logs. The columns in amber are the ones added for the job rules.

## 5.1 What each table stores

| Table | Purpose |
|---|---|
| **Roles** | The two roles: Candidate and Recruiter. |
| **Users** | Every account: name, email, password hash (never the password), role, active flag, and the code and expiry of the current one-time code. |
| **Candidates / Recruiters** | The extra details of each role: phone, address and date of birth for candidates; company name and website for recruiters. One row per user. |
| **Resumes** | Each uploaded file: name, path on the server, the text the AI read (ParsedText) and the upload time. |
| **Skills, CandidateSkills, JobSkills** | A list of skill names, and the links between skills and candidates or jobs. |
| **CandidateEducation, CandidateWorkExperience** | Education and work history found in the resume. |
| **Jobs** | A job post: recruiter, title, description, location, status (Open or Closed), post date, and the five rule columns: ExclusiveApplication, HireLockMonths, HireLockScope, RejectWaitDays, ResponseDeadlineDays, plus PoliciesAcceptedAt. |
| **Applications** | One application: job, candidate, resume, status, MatchScore, AppliedAt, ShortlistedAt, DecidedAt (when hired or rejected) and AcceptedPoliciesAt. |
| **InterviewStatus** | The six stage names, used for both applications and interviews: Applied, Shortlisted, Scheduled, Completed, Rejected, Hired. |
| **Interviews** | Date and time, mode (online or in person), venue and the recruiter's comments for an application. |
| **AuditLog** | The activity trail: which record changed (table name and record id), what happened, who changed it, when, and the old and new values. It feeds both Activity feeds. |
| **ConsentLog** | Proof that the Terms of Service and the Privacy Policy were accepted: which document, which version, when, and by whom (account or anonymous visitor id). |

## 5.2 How the database is changed safely

- Structure changes are written as **EF Core migrations**. When the API starts it applies any migration that is missing, so a deploy updates the database by itself.
- New columns are added empty or with a default, so old data stays valid.
- For risky changes the order is: frontend first, then the database columns, backend merge last. This way nothing breaks half way.
- Backups: the SQL_server folder holds the schema script, test scripts and a backup script (backup_database.sh).

# 6 The AI service and the match score

The AI service is a small Python program (FastAPI) that runs next to the API. It has two helpers: a **language model** (Groq) that lists skills, and a small **embedding model** (MiniLM) that turns text into numbers so meanings can be compared.

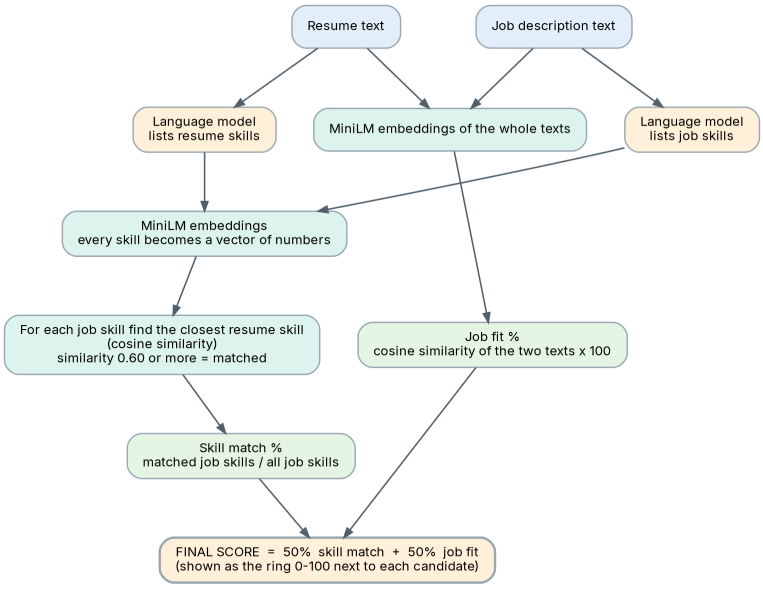



Figure 5. How a score is made.

## 6.1 The formula

| Part | How it is calculated |
|---|---|
| **Skill match %** | The language model lists the skills in the resume and in the job. Every skill becomes a vector. For each job skill the closest resume skill is found (cosine similarity). A similarity of 0.60 or more counts as a match. Skill match = matched job skills / all job skills x 100. |
| **Job fit %** | The whole resume text and the whole job text are turned into vectors. Their cosine similarity x 100 tells how close the two documents are. |
| **Final score** | 50% Skill match + 50% Job fit. Example: 83% and 49% give 66%. |

## 6.2 Endpoints of the AI service (private, API key required)

| Endpoint | Used for |
|---|---|
| **POST /extract-skills** | Reads one resume and returns its skill list (used when a resume is uploaded). |
| **POST /resume-summary** | Returns structured data from a resume: education, experience and similar fields. |
| **POST /match-job and /candidate-score** | Compare a resume file with a job description file (testing and reports). |
| **POST /score-application** | Same score, but the job text comes from the database. This is what the API calls when a candidate applies. |
| **GET /** | Health check. |

**Fairness note.** The score never hires or rejects anyone. A recruiter sees the numbers and the matched skills and decides. Candidates can see their own score.

# 7 Security, privacy and rules

| Feature | How it works |
|---|---|
| **Password storage** | Passwords are hashed with BCrypt. The database never holds the real password. |
| **Login token** | JWT bearer token, valid for 8 hours. The API rejects calls with a missing or expired token (401). |
| **Own data only** | Profile, resume and application endpoints compare the id in the address with the id in the token (403 if different). |
| **One-time codes** | A 6-digit code with an expiry, sent by email. It protects sign-up, forgot password, change password and delete account. |
| **Human check** | Cloudflare Turnstile on login, both sign-ups, forgot password and resend code. The API checks the token again on the server. |
| **Rate limits** | RateLimitService slows down repeated requests (for example many login or code attempts). |
| **Google sign-in** | The API validates the Google ID token itself. Google accounts get a random password they never use. |
| **Consent** | Sign-up needs a tick for the Terms and the Privacy Policy. The tick, its version and its time go to ConsentLog. A first-visit notice records acceptance of local storage. |
| **Delete account** | Needs the emailed code. It removes the account, resumes (including the files), applications and, for recruiters, jobs and interviews. |
| **Job rules** | Enforced in the API, not only on the screen: a candidate cannot apply without accepting the rules, or if a rule blocks it. |
| **Transport** | HTTPS everywhere. The API redirects HTTP to HTTPS. |

## 7.1 The four job rules a recruiter can choose

| Rule | What it means for the candidate |
|---|---|
| **One job at a time** | If on, a candidate with an open application to another job of the same recruiter cannot apply to this job. |
| **Pause after a hire** | If a candidate is hired, they cannot apply to other jobs of that recruiter (or of that company) for 6, 12, 24 months or a number the recruiter sets. |
| **Wait after a rejection** | After a rejection the candidate must wait the chosen number of days before applying to the same job again. The message names the exact date. |
| **Reply promise** | The recruiter promises to answer within the chosen number of days. The candidate sees the countdown and the recruiter sees the late replies in red. |

# 8 Files and folders

Today (4 November 2026) the repository tracks **313** files: 159 C#, 89 React and JavaScript, 5 SQL; 11 controllers, 39 services, 31 repositories, 49 models, 4 migrations, 16 pages and 47 test files. The first version of the project folder (a snapshot of 6 October 2026) held **190** hand-written files. Many were added after that (consent, human check, job rules, emails, profile, dashboard polish and their tests). The folder tree below lists what each part is for. To count today's exact numbers, run the command in section 8.3.

## 8.1 The folder tree

| Path | What is inside | Files (6 Oct) |
|---|---|---|
| **BackendAPI/** | The web layer: Program.cs (start-up and settings), appsettings, Controllers (Auth, Users, Password, Jobs, Applications, Resumes, Interviews, Dashboard, AiMatching, Roles, Consent), launch settings, uploads folder for resume files | 15 |
| **BackendAPI.Core/Models** | Entities and request/response objects (User, Candidate, Recruiter, Resume, Job, Application, Interview, AuditLog, login and register requests ...) | 45 |
| **BackendAPI.Core/Repository** | ApplicationDbContext and one repository (with its interface) per table | 29 |
| **BackendAPI.Core/Services** | Business rules: Auth, Otp, Jwt, User, Job, Application, Resume, Interview, Dashboard, AiMatching, AiServiceClient, Email, RateLimit (each with an interface) | 28 |
| **BackendAPI.Core/Migrations** | Database migrations in order | 5 |
| **BackendAPI.Tests/** | xUnit tests | 4 |
| **Frontend/src/pages** | One file per screen: Landing, Login, Register, OTP verify, Forgot and Reset password, Dashboard, Resume, Candidate and Recruiter jobs, Activity feeds, Analytics, Profile, Privacy and Terms, Not found | |
| **Frontend/src/components** | Reusable parts: Navbar, Footer, Layout, ProtectedRoute, the cards and badges (ui.jsx), StatTile, HeroOrb, QuickStart, DashboardParts, JobRules, TurnstileBox, ConsentBanner, FieldError | |
| **Frontend/src/api** | One file per API area (authApi, userApi, jobApi, resumeApi, interviewApi, dashboardApi) and axiosClient, which adds the login token and handles a 401 | |
| **Frontend/src/context, lib, utils, styles** | Login state (AuthContext), helper hooks and icons, validation and formatting helpers, and the style sheets (tokens.css for colours and fonts) | |
| **Frontend (total)** | Website code, tests, public files (logo, robots.txt, sitemap.xml, social image) | 43 |
| **AiService/** | main.py (the whole service), Dockerfile, requirements.txt | 4 |
| **SQL_server/** | Schema script, test and patch scripts, backup script | 6 |
| **.github/workflows** | ci.yml (tests on every push and pull request) and the backend deploy workflow | 1 |
| **Postman/, scripts/** | A Postman collection to try the API, and a warm-up script | 2 |

## 8.2 Frontend routes

| Address | Who | Screen |
|---|---|---|
| / | Everyone | Landing page |
| /login, /register | Visitors | Log in, Create account |
| /verify-otp, /forgot-password, /reset-password | Visitors | Enter the emailed code, ask for a code, set a new password |
| /privacy, /terms | Everyone | Privacy Policy, Terms of Service |
| /dashboard, /profile | Logged in | Dashboard (different for each role), Profile |
| /resumes, /jobs, /my-activity | Candidate | My Resumes, Browse Jobs, My Activity |
| /my-jobs, /analytics, /activity-feed | Recruiter | My Jobs, Analytics, Activity Feed |
| Any other address | Everyone | Page not found (404) with a way back |

## 8.3 Count the files today

Run this in the project folder to get the exact numbers:

```
git ls-files | wc -l
git ls-files "*.cs" | wc -l
git ls-files "*.jsx" | wc -l
```

# 9 Flows with arrows

## 9.1 Sign-up and email verification

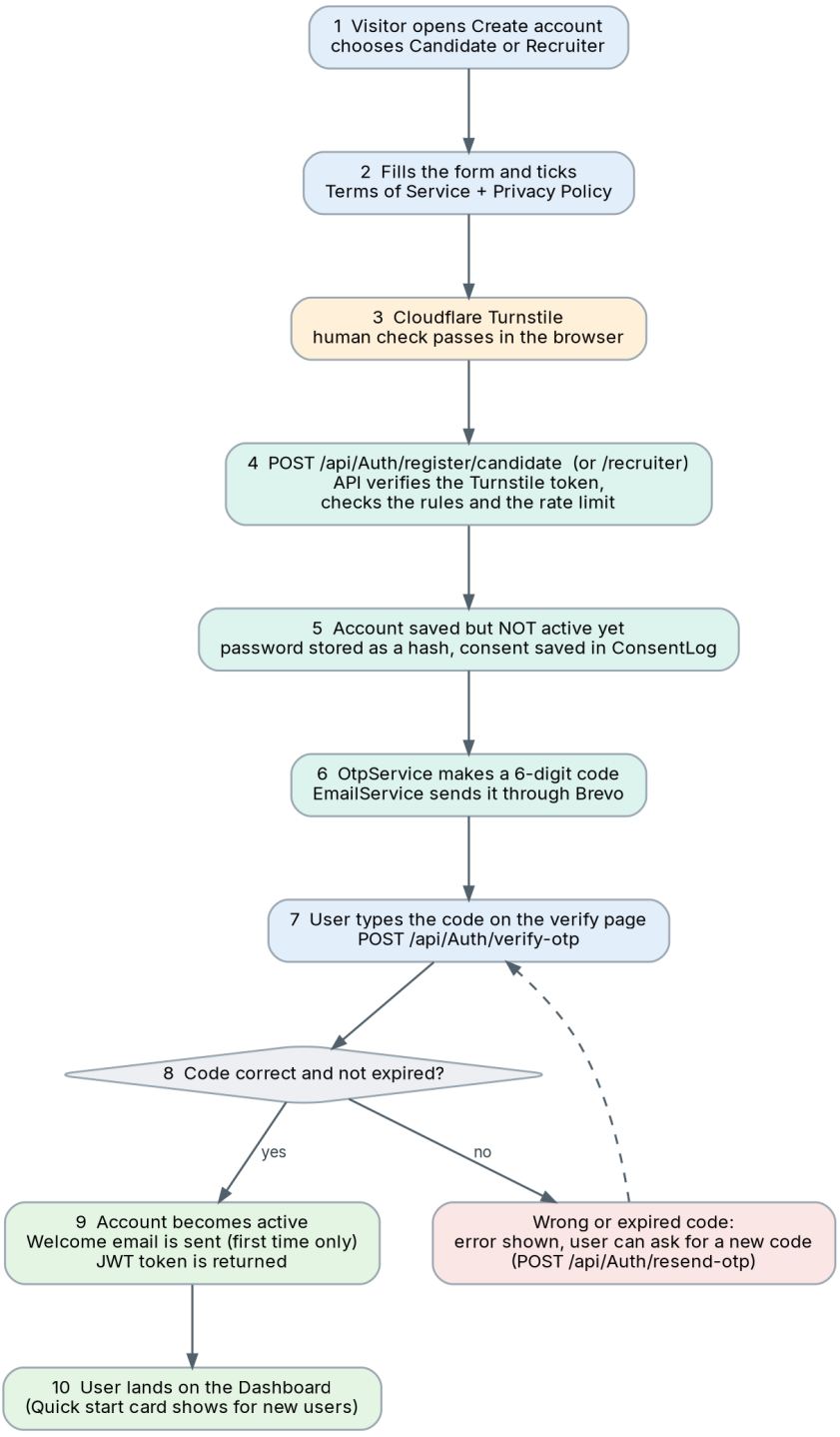



Figure 6. A new account.

## 9.2 The one-time code, used four ways

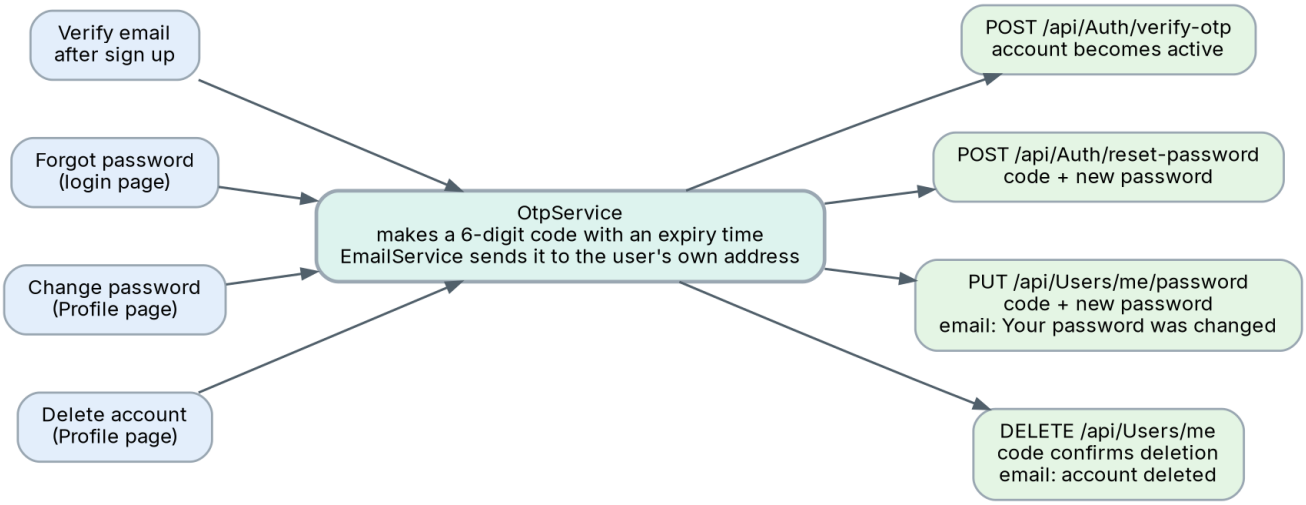



Figure 7. One service makes and sends the code, four screens use it.

## 9.3 Upload and analyse a resume

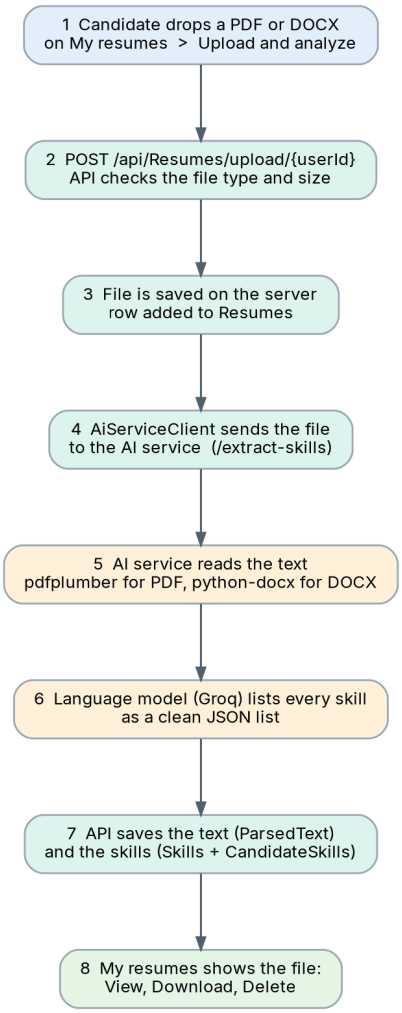



Figure 8. Resume upload.

## 9.4 Apply to a job

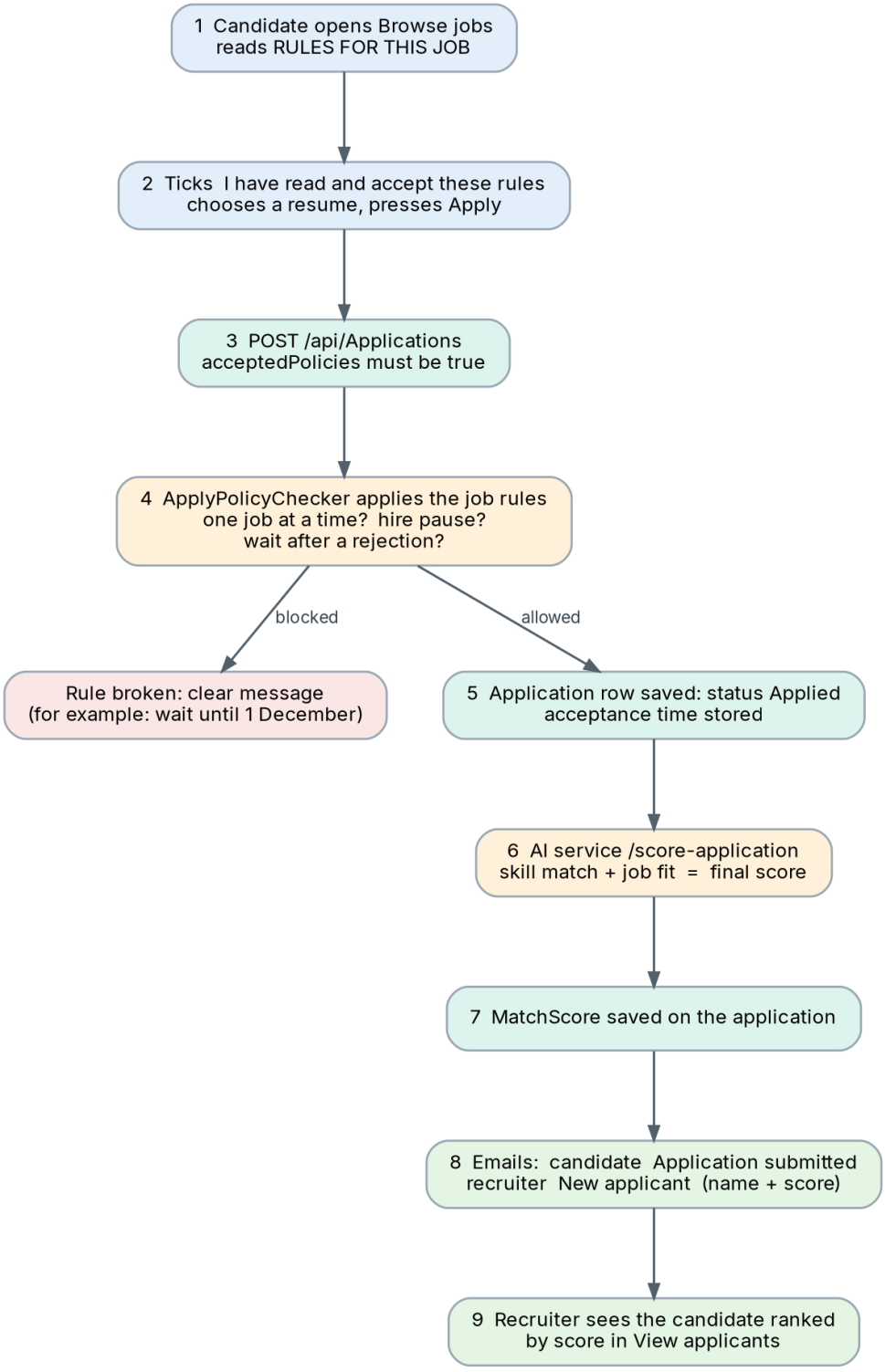



Figure 9. Applying, with the job rules and the score.

## 9.5 The hiring pipeline

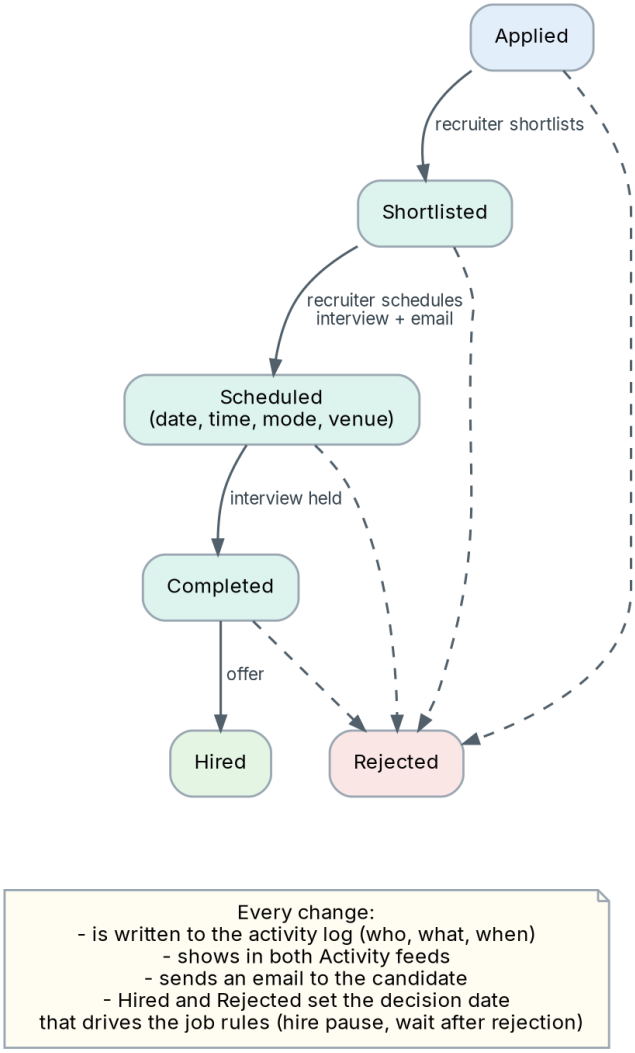



Figure 10. Stages of an application.

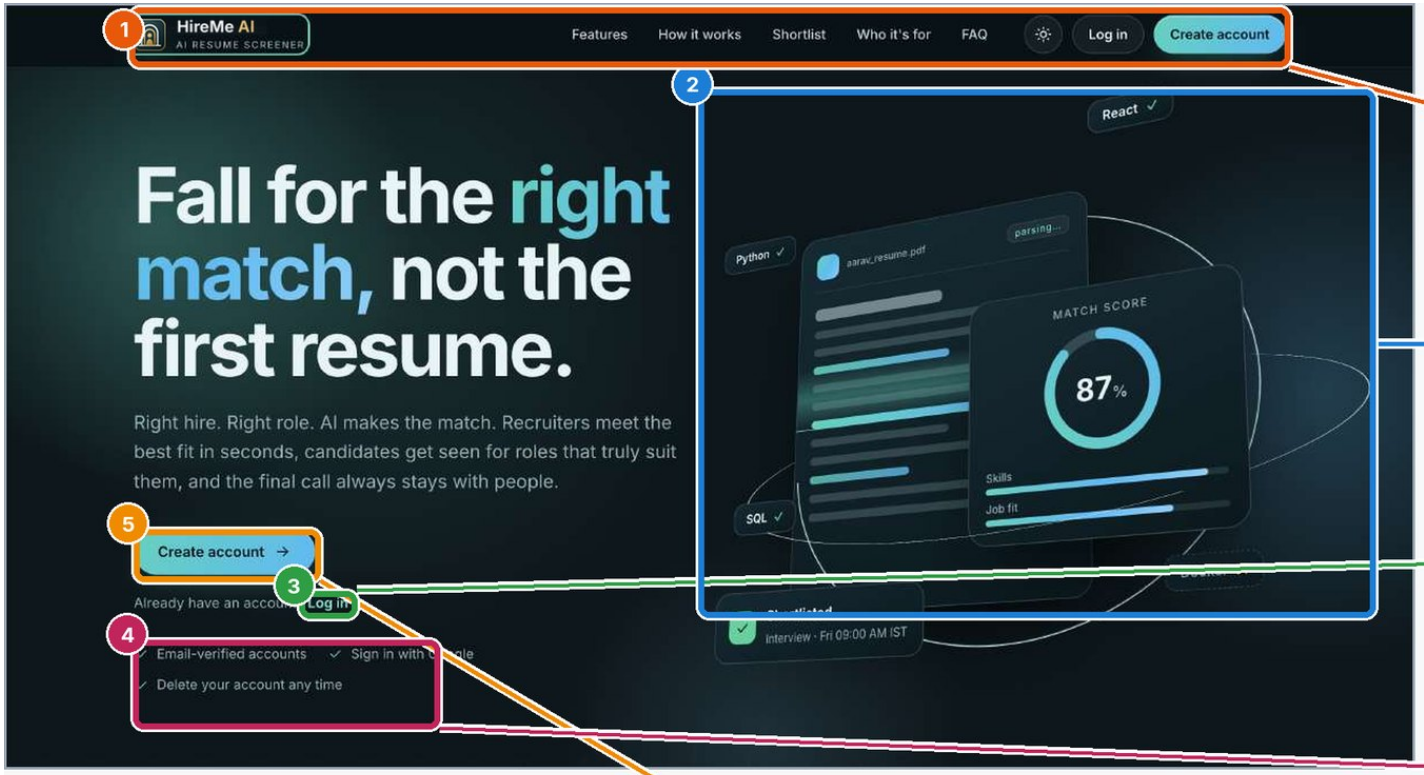
- A recruiter moves an application with **PUT /api/Applications/{id}/status**. Scheduling an interview is a separate call: **POST /api/Interviews**.
- Every change writes an AuditLog row, shows in both Activity feeds and sends the candidate an email.
- Hired and Rejected store the decision time (DecidedAt). The job rules use it: the pause after a hire and the wait after a rejection are counted from that moment.

# 10 Working guide: pages everybody sees

These are the pages before login. The numbered boxes follow the numbers on the screenshot.

## Landing page: first screen

What a visitor sees first. Nothing here needs an account.




**1 Top bar**
Features, How it works, Shortlist, Who it is for and FAQ scroll the page to that part. The sun / moon button switches between light and dark theme.

**2 Live preview of the product**
A moving picture of how it works: a resume is read, skills such as React and SQL are found, the AI match score (87%) is shown and a Shortlisted card with an interview time appears.

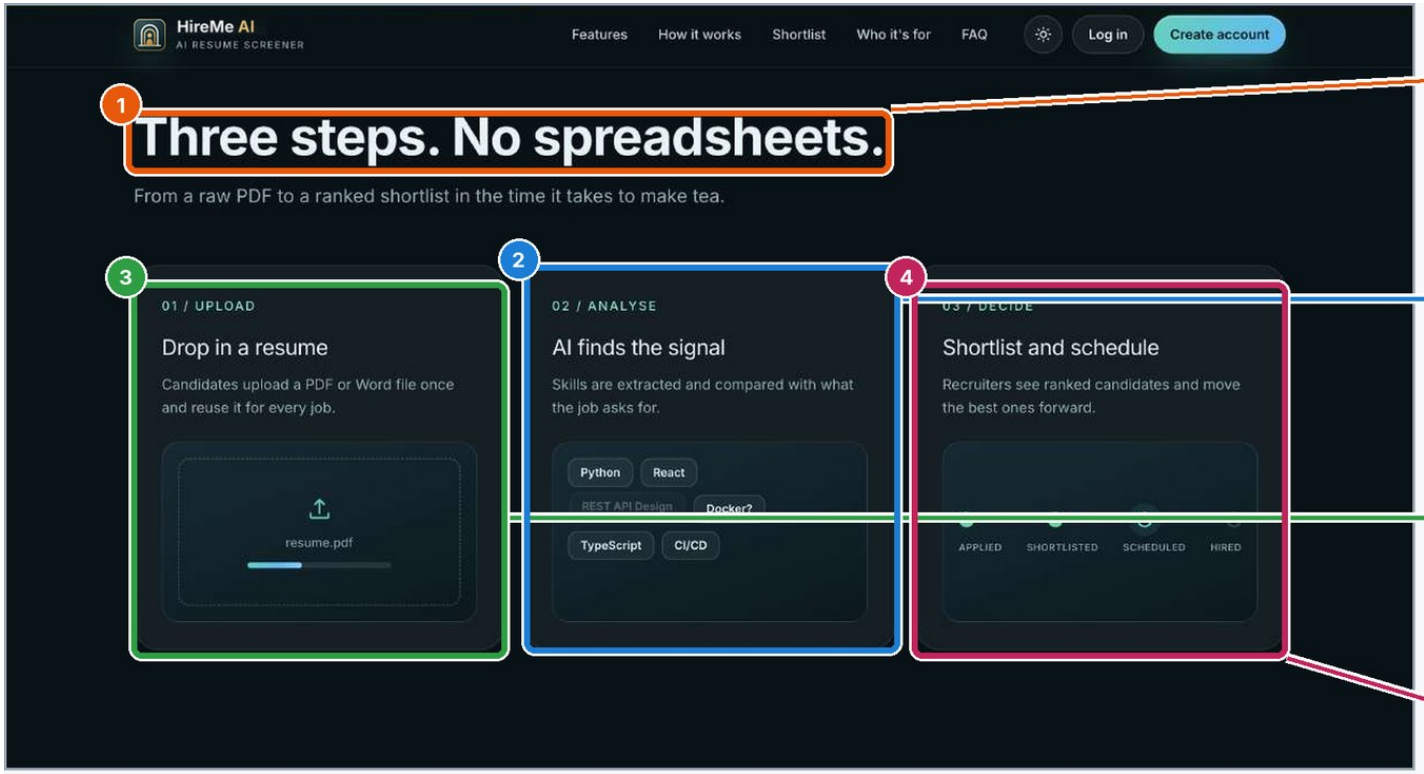
**3 Log in link**
People who already have an account log in from here. The text under the big button says it in words: Already have an account? Log in.

**4 Trust points**
Three promises: accounts are verified by email, Google sign-in is available, and a user can delete the account at any time.

**5 Create account**
The main call to action. It opens the free sign-up for a candidate or a recruiter. The same button is in the top bar.

## Landing page: how it works

The three steps of the product, shown as small pictures.




**1 Headline**
Says what the product replaces: manual sorting of resumes in spreadsheets.

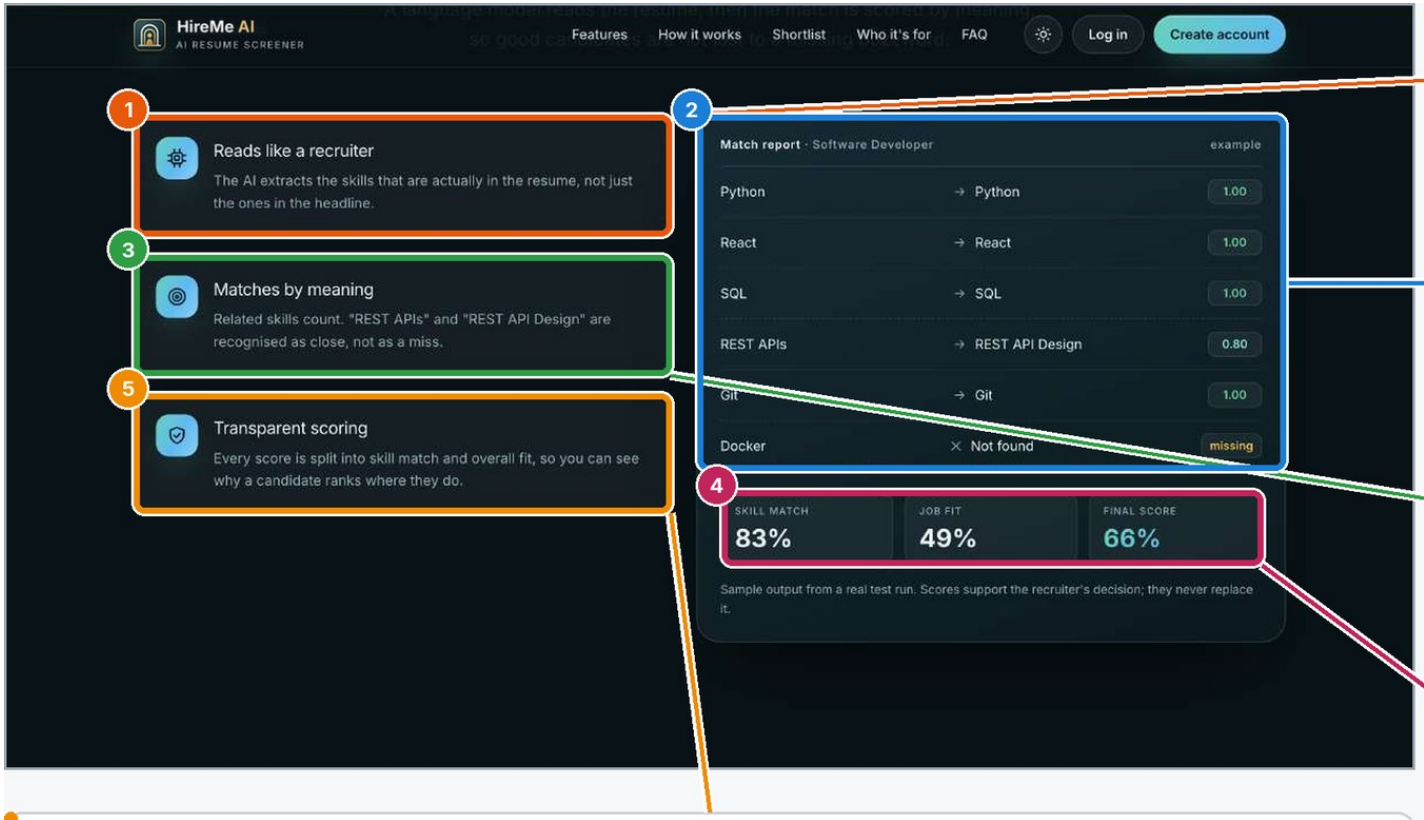
**2 Step 2: Analyse**
The AI pulls the skills out of the resume (Python, React, REST API Design ...) and compares them with what the job asks for. Skills the job needs but the resume lacks show as missing.

**3 Step 1: Upload**
A candidate uploads a PDF or Word resume once and can use it for every job.

**4 Step 3: Decide**
Recruiters see the ranked candidates and move the best ones along the stages: Applied, Shortlisted, Scheduled, Hired.

# Landing page: how the match is explained

A sample match report. It shows that the score is split into parts a recruiter can understand.




**1 Reads like a recruiter**

The AI extracts the skills that are really in the resume, not only the ones in the headline.

**2 Match report table**

Each skill the job needs is paired with the closest skill on the resume and gets a similarity from 0 to 1. Docker was Not found, so it is marked missing.

**3 Matches by meaning**

Related skills count: REST APIs on a job still matches REST API Design on a resume, so good people are not lost to a missing keyword.

**4 Three numbers**

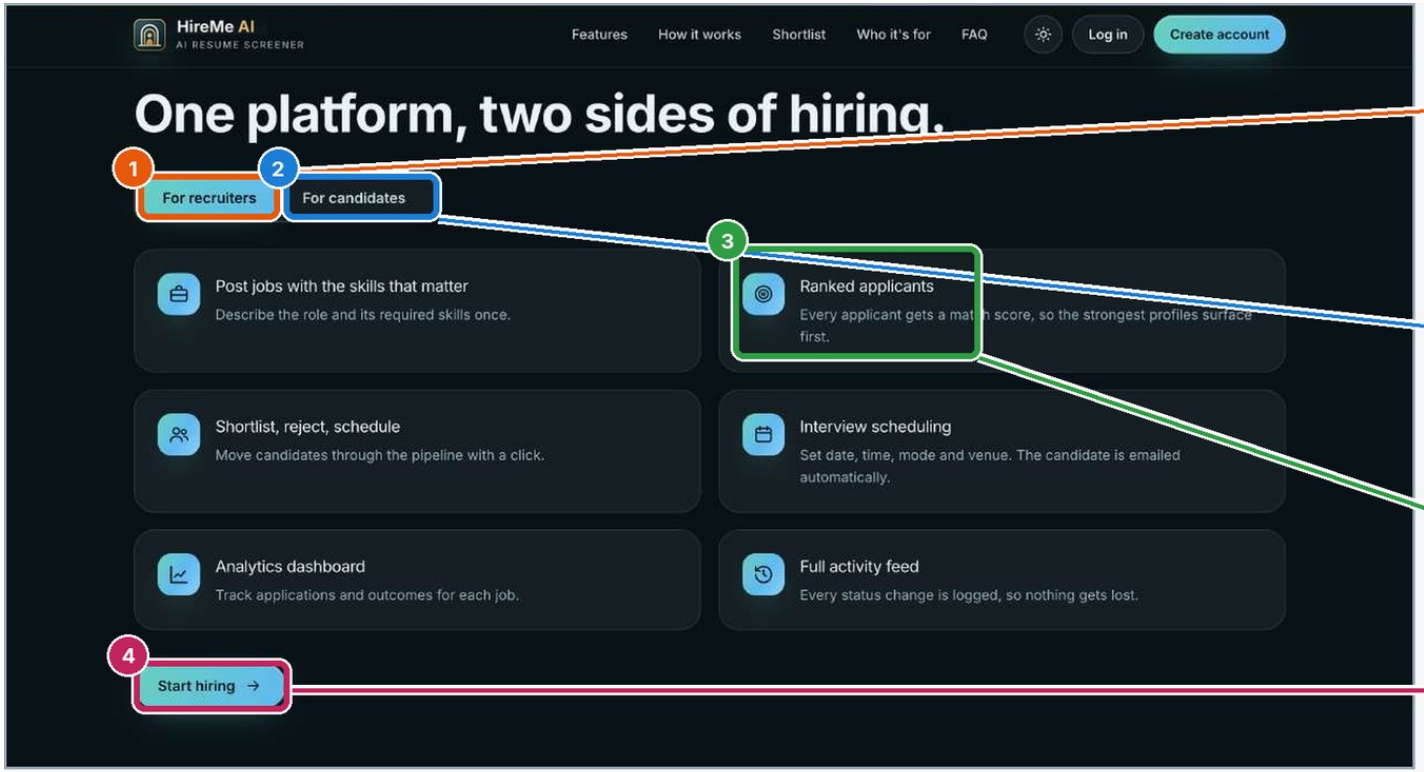
Skill match (83%) = share of job skills found.
Job fit (49%) = how close the whole texts are.
Final score (66%) = half of each.

**5 Transparent scoring**

Every score can be explained, so a recruiter can see why a candidate ranks where they do. The score supports a decision, it never replaces it.

## Landing page: two sides of hiring

The product has one workspace for recruiters and one for candidates.




**1 Recruiter tab**

Shows what a recruiter gets: post jobs with the skills that matter, ranked applicants, shortlist / reject / schedule, interview emails, analytics and a full activity feed.

**2 Candidate tab**

Click it to read the candidate side: upload a resume once, apply to jobs, see the AI match score and follow every status.

**3 Ranked applicants**

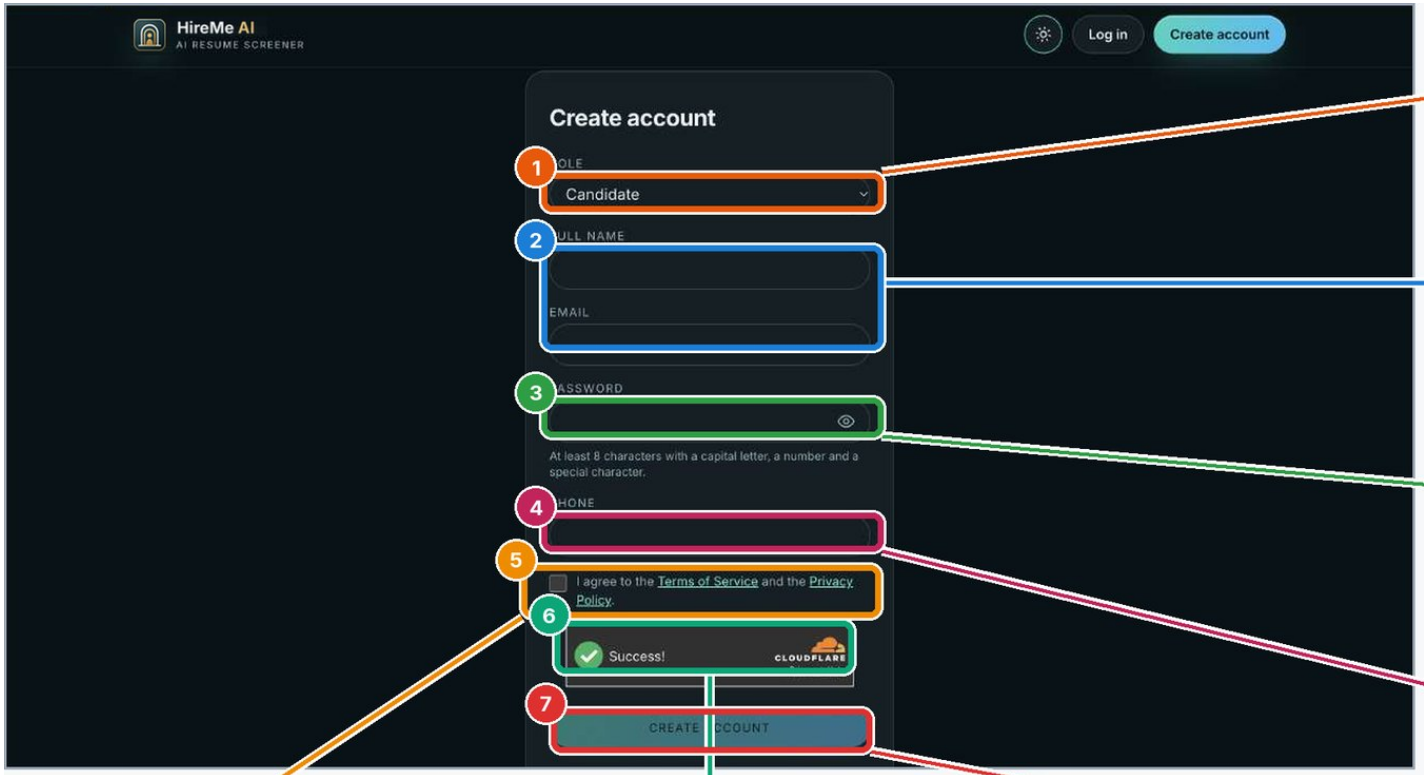
Every applicant gets a match score, so the strongest profiles come first.

**4 Start hiring**

A second call to action at the end of the section. It opens the sign-up.

# Create account (top of the form)

One form for both roles. The role you pick decides which fields you see.




**1 Role**
Choose Candidate or Recruiter. A recruiter is also asked for the company name. The role cannot be changed later.

**2 Full name and email**
The email must be real: a one-time code is sent to it to confirm the account.

**3 Password with eye icon**
The eye icon shows or hides what you type. The rule is written below the box: at least 8 characters with a capital letter, a number and a special character.

**4 Phone**
Candidates give a 10-digit mobile number. Only digits are accepted.

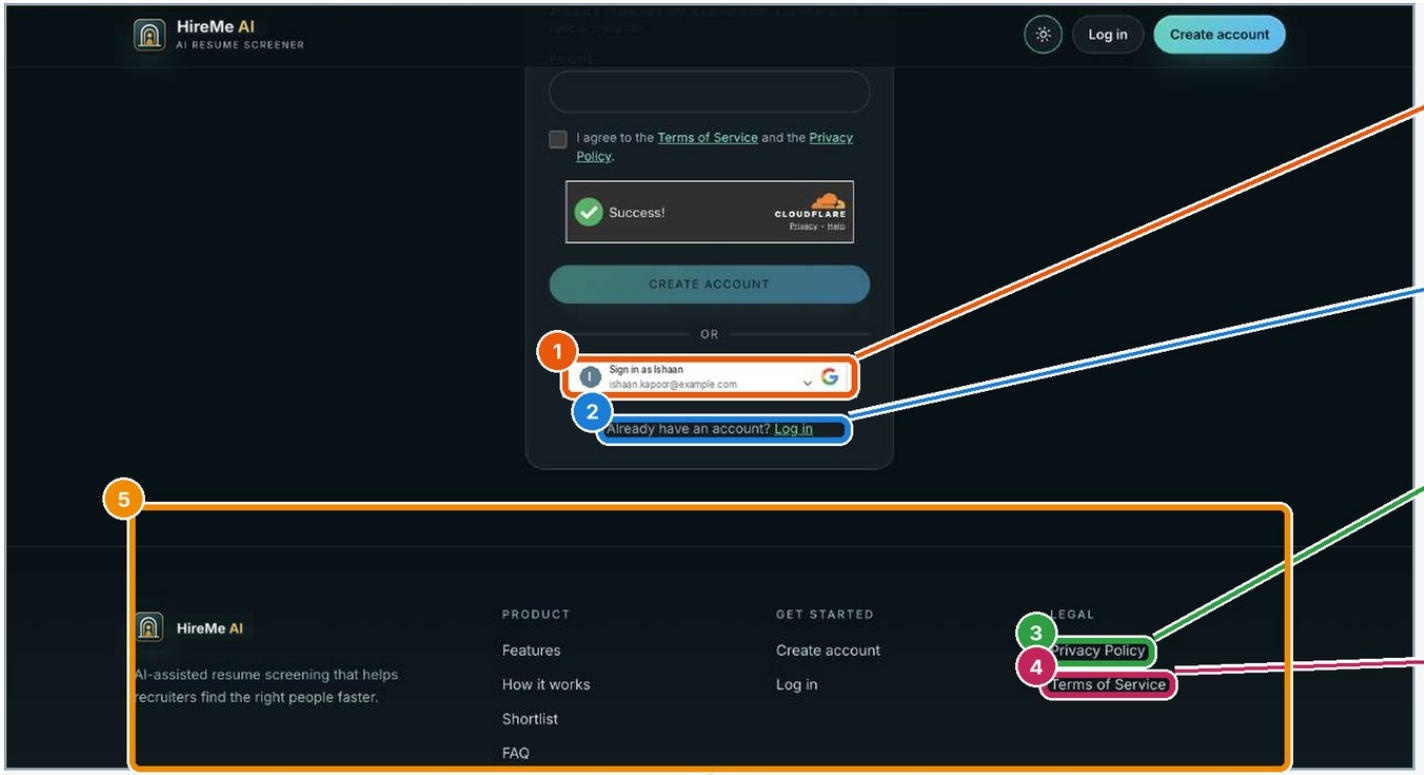
**5 Terms and Privacy**
The account cannot be created until this box is ticked. The links open the Terms of Service and the Privacy Policy. The tick and its version are saved.

**6 Human check**
Cloudflare Turnstile checks that a person, not a bot, is signing up. It is invisible most of the time.

**7 Create account button**
Stays faded until the form is complete and the box is ticked.

# Create account (bottom) and footer

The other ways to start, and the legal links.




**1 Sign in with Google**
A quicker way in. Google confirms the email, so no code is needed. New Google users are asked to pick Candidate or Recruiter first.

**2 Already registered**
The Log in link takes existing users to the login page.

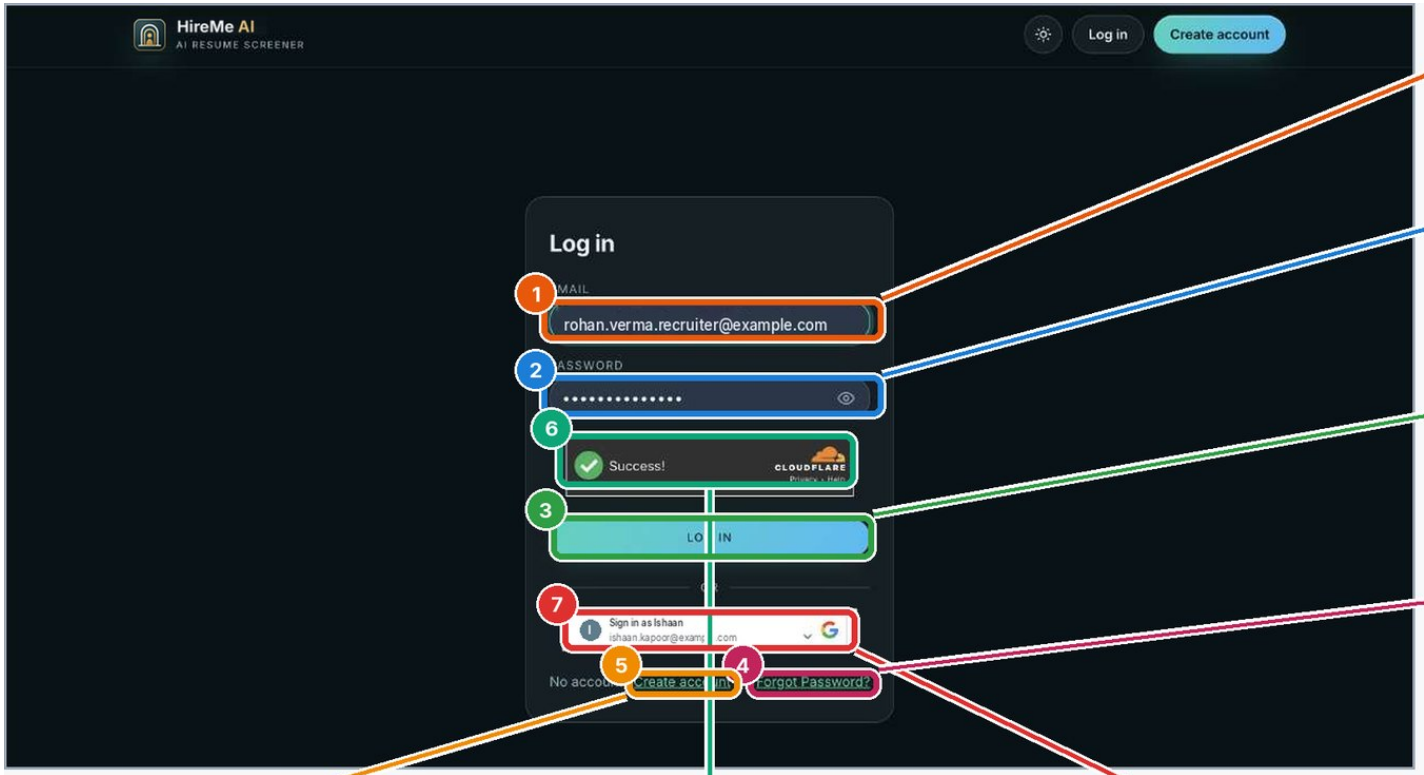
**3 Privacy Policy**
Explains what data is collected, who processes it and how to delete it. It is in the footer of every page.

**4 Terms of Service**
The rules for using the platform as a candidate or a recruiter.

**5 Footer**
Product links, a Get started column (Create account, Log in) and a Legal column. There is no copyright line.

# Log in

The same screen for candidates and recruiters.




**1 Email**
The email used at sign-up.

**2 Password with eye icon**
Click the eye to see the password and check the typing.

**3 Log in**
Checks the email and password. On success the server sends a login token (JWT, valid for 8 hours) and the user lands on the Dashboard.

**4 Forgot Password?**
Sends a 6-digit code to the email. With the code the user sets a new password.

**5 Create account**
For people with no account yet.

**6 Human check**
Turnstile must show Success before the button works.

**7 Sign in with Google**
Logs in with the Google account. If the email is new, the user is asked to choose a role.

# 11 Other pages for visitors

These pages are not in the screenshot set, so they are described in words.

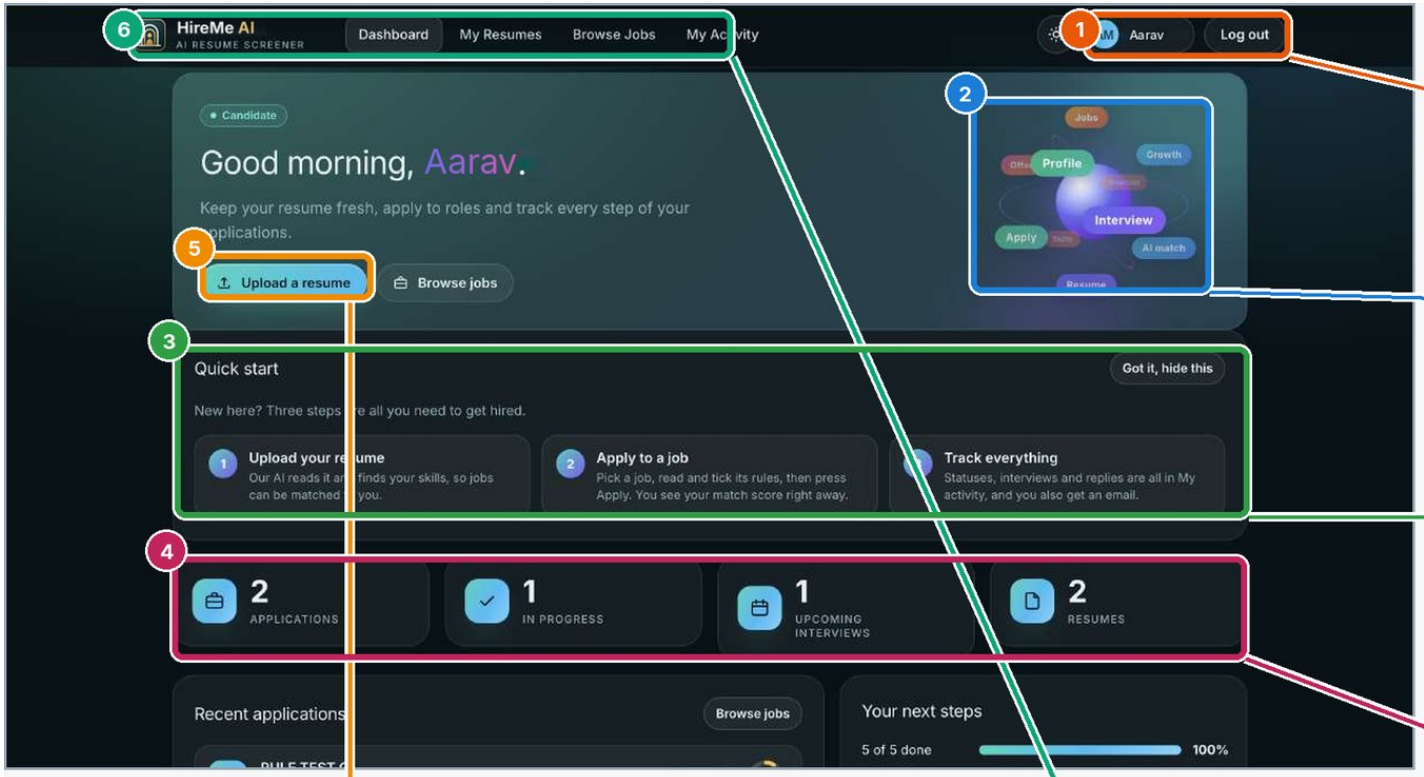
| Page | What it does |
|---|---|
| **Verify your email** | Opens after Create account. The user types the 6-digit code from the email. A Resend code link asks for a new code. After success the account is active, a welcome email is sent and the user lands on the Dashboard. |
| **Forgot password** | Asks for the email (and a human check), sends a code. The page then opens Reset password. |
| **Reset password** | Takes the email, the code and the new password (typed twice, with an eye icon). Success sends the person to Log in. |
| **Privacy Policy and Terms of Service** | Plain-language legal pages with a contents list on the left, a date of the last update and the contact email of the project. |
| **Page not found** | Shown for any wrong address, with a button back to the home page. |
| **First-visit notice** | A small banner at the bottom says that the site stores a login token in the browser. OK closes it and the choice is remembered. |
| **Light and dark theme** | The sun and moon button in the top bar switches the colours and remembers the choice. |
| **Phone menu** | On a small screen the top bar becomes a menu button that opens the same links, plus theme, Profile and Log out. |
| **Scroll progress bar** | A thin line under the top bar shows how far down the page you are. |

# 12 Working guide: the candidate

Follow the screens in the order a candidate meets them: Dashboard, resume, jobs, profile.

## Candidate: Dashboard (top)

The first page after login. It tells a new candidate what to do and shows the numbers that matter.




**1 Profile and log out**
Click the name to open the Profile page (details, change password, delete account). Log out ends the session.

**2 Skill orb**
A 3D ball of labels (Resume, Skills, AI match, Interview ...). Drag it with the mouse to turn it. It is only for fun and explains nothing by itself.

**3 Quick start**
Three clear steps for new users: upload a resume, apply to a job, track everything. Got it, hide this closes the card for good, and Show quick start brings it back.

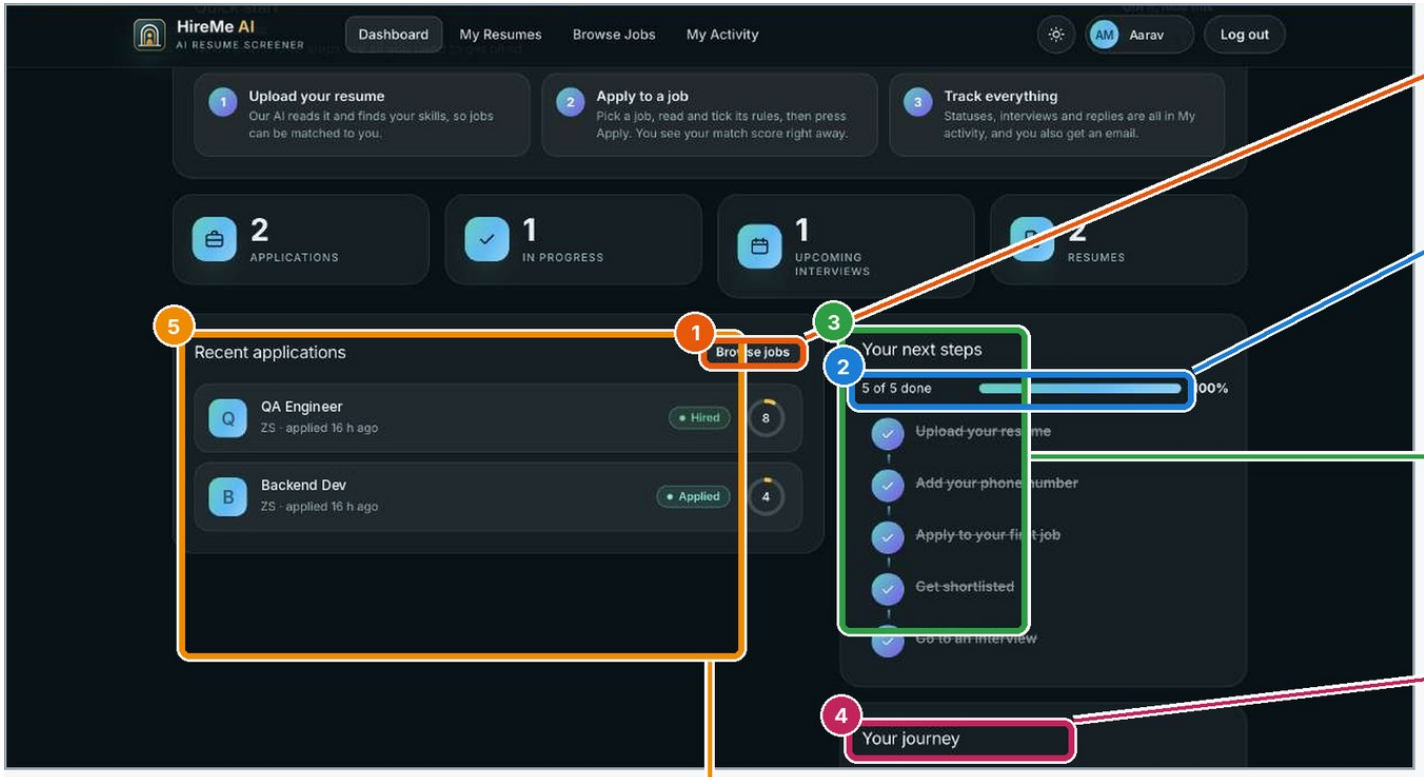
**4 Number cards**
Applications, In progress, Upcoming interviews, Resumes. Point at a card to read what it means, click it to go to the right page.

**5 Upload a resume**
Goes straight to My Resumes. This is step one for every candidate.

**6 Candidate menu**
Dashboard, My Resumes, Browse Jobs and My Activity. The page you are on is highlighted.

# Candidate: Dashboard (lower part)

Where the candidate follows applications and sees what to do next.




**1 Browse jobs**
Shortcut to the list of open jobs.

**2 Progress bar**
Shows how many steps are done. 100% means the candidate has completed the whole journey.

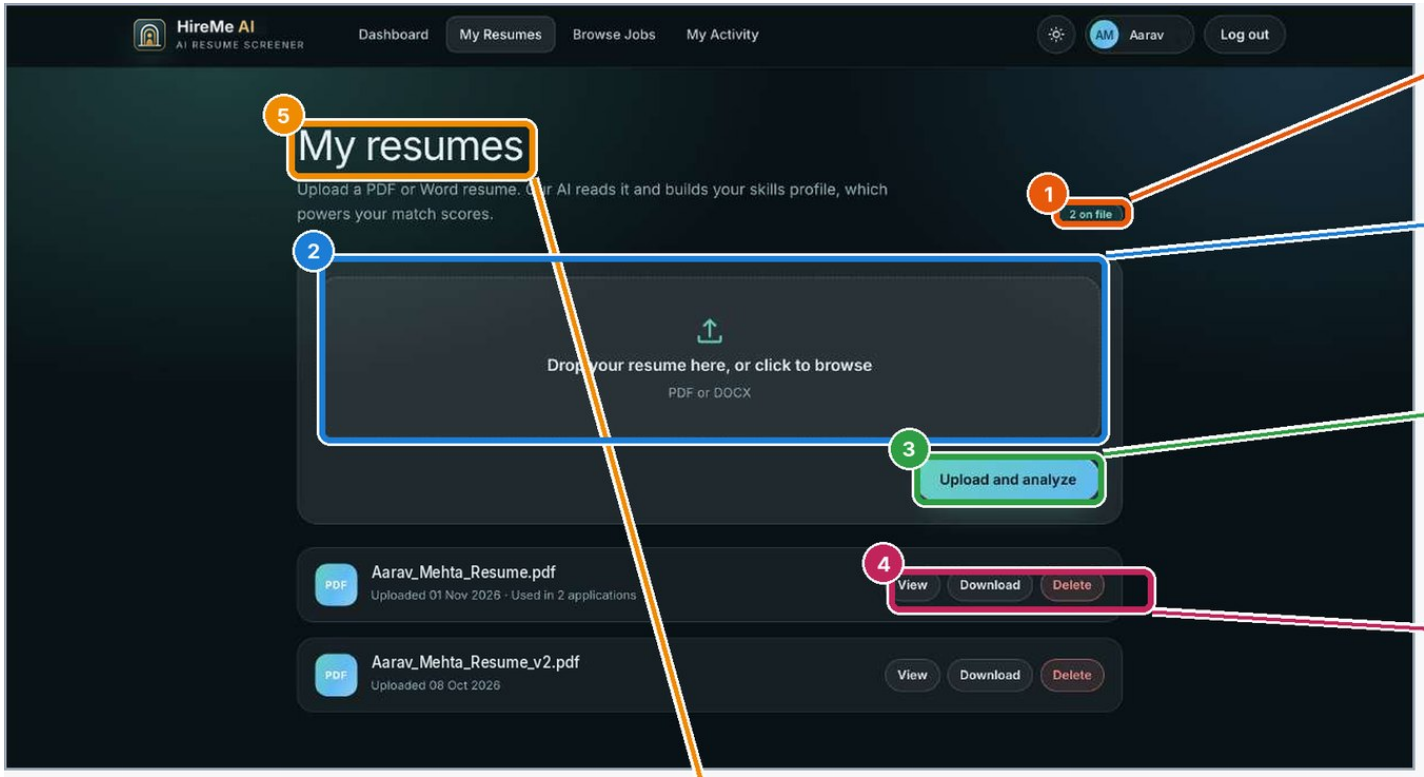
**3 Your next steps**
A guide with five steps: upload a resume, add a phone number, apply, get shortlisted, go to an interview. Finished steps get a tick and a line through them.

**4 Your journey**
Bars that count applications at each stage: Applied, Shortlisted, Interview, Hired, Not selected.

**5 Recent applications**
The latest applications with the company, how long ago the candidate applied, the status badge (Applied, Hired ...) and the AI match score as a ring.

# Candidate: My resumes

Resumes are uploaded here once and can be used for every job.




**1 Resumes on file**
How many resumes are saved for this account.

**2 Upload area**
Drag a PDF or DOCX file here or click to browse. Other file types are refused.

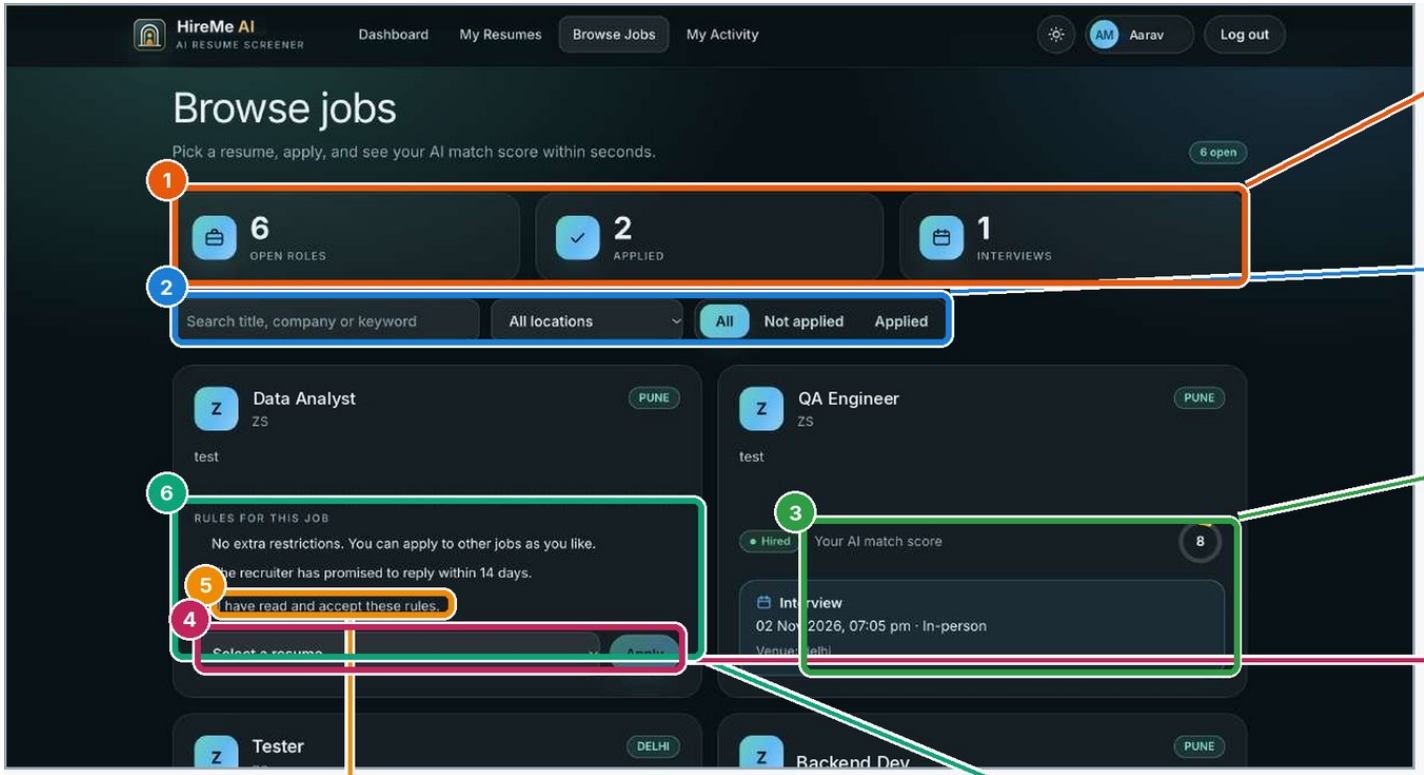
**3 Upload and analyze**
Sends the file to the server. The AI service reads the text, finds the skills and saves them. It takes a few seconds.

**4 View, Download, Delete**
View opens the file in the page, Download saves a copy, Delete removes the resume. Under the name, the page tells how many applications used this resume.

**5 My resumes**
Upload and manage resumes. The AI reads each one and builds the skills profile that powers the match scores.

# Candidate: Browse jobs

Every open job, the rules of each job and one-click apply.




**1 Number cards**
Open roles, Applied, Interviews. Open roles and Applied also work as filters for the list below.

**2 Search and filters**
Search by title, company or keyword. Choose a location. The tabs All, Not applied and Applied narrow the list.

**3 After applying**
The card shows the status (here Hired), the match score ring and, when a recruiter sets one, the interview date, time, mode and venue.

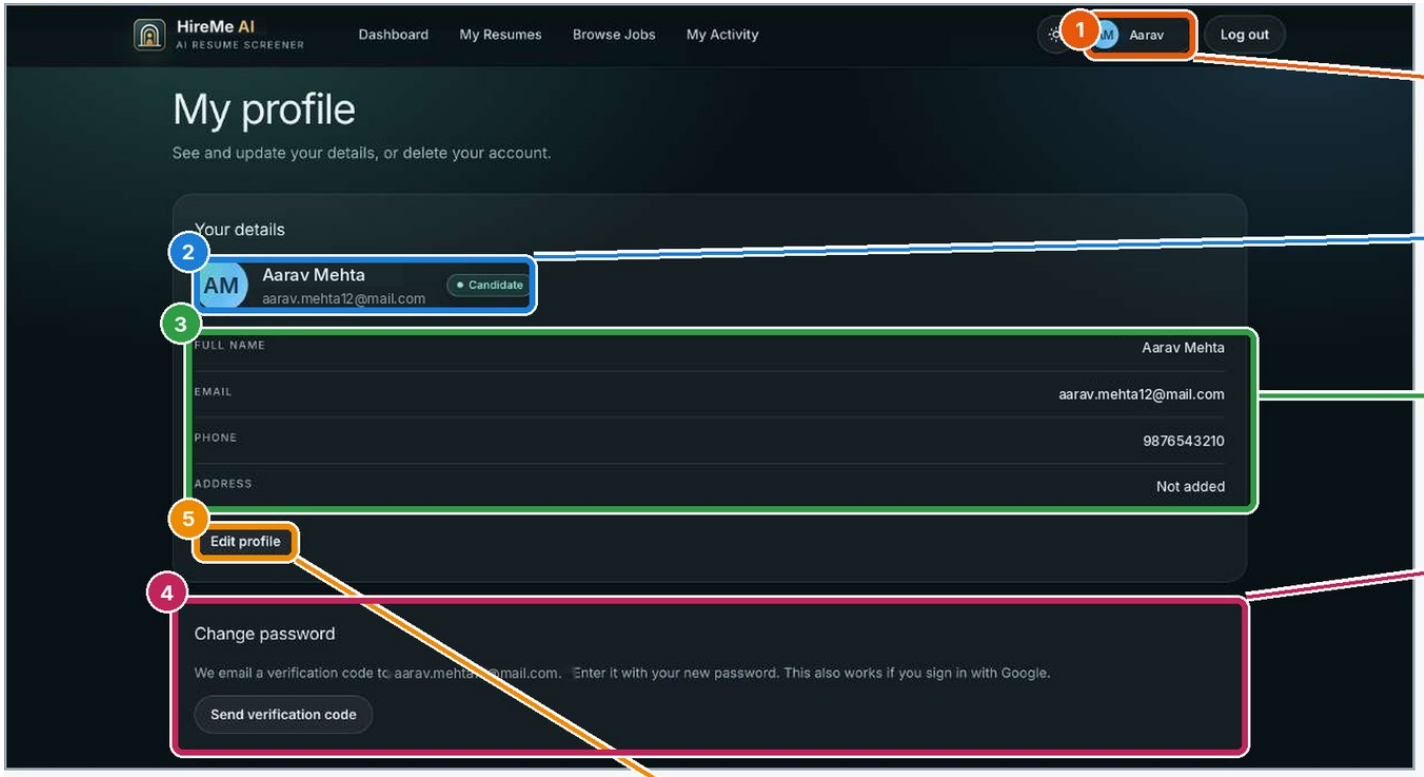
**4 Choose a resume and Apply**
Pick one of the saved resumes and press Apply. The AI score appears within seconds.

**5 Accept the rules**
Apply stays off until this box is ticked. The time of acceptance is saved with the application.

**6 Rules for this job**
The recruiter's conditions are shown before applying, for example: no extra restrictions, or the recruiter promises to reply within 14 days.

# Candidate: Profile page

Opened by clicking the name in the top bar.




**1 Profile button**
Click your name to open this page from anywhere.

**2 Account summary**
Name, email and role (Candidate).

**3 Your details**
Full name, email, phone and address. The email cannot be edited.

**4 Change password**
Passwords are changed here, not on the Dashboard.

**5 Edit profile**
Opens a small form for the name, phone (10 digits) and address. The new name appears in the top bar at once.

# Candidate: change password and delete account

Both actions are protected by a code sent to the account email.




**1 Change password**
Press Send verification code. A 6-digit code arrives by email. Enter it with the new password twice. This also works for people who sign in with Google. A confirmation email is sent afterwards.

**2 Send verification code**
Starts the change. Nothing changes until the correct code is entered.

**3 Danger zone**
The red box warns that deleting the account is permanent: the account, resumes and all applications are removed.

**4 Delete Account**
Sends a code to the email. The account is deleted only after the code is entered. A goodbye email confirms it.

## 12.1 Everything a candidate can do

| Function | Where | How it works |
|---|---|---|
| **Create an account** | Create account | Email, password, phone, tick for Terms and Privacy, human check, then a 6-digit code by email. A welcome email follows. |
| **Sign in with Google** | Login / Create account | Google proves the email. A new user picks the role first. |
| **Log in, forgot and reset password** | Login | A wrong password gives a short message; Forgot Password sends a code. |
| **Quick start and Your next steps** | Dashboard | Three steps for new users, and a five-step progress guide that ticks itself (resume, phone, apply, shortlisted, interview). |
| **Number cards** | Dashboard, Browse jobs | Each number explains itself when you point at it for a moment and opens the right page when you click. |
| **Upload a resume** | My Resumes | PDF or DOCX. The AI reads it, finds the skills and shows it in the list. Several resumes can be kept. |
| **View, download, delete a resume** | My Resumes | View opens the file, Download saves it, Delete removes it. The list says how many applications used it. |
| **Browse and filter jobs** | Browse Jobs | Search by title, company or keyword; filter by location; tabs All, Not applied, Applied. |
| **Read and accept the job rules** | Browse Jobs | The rules are on the job card. Apply stays off until the box is ticked. |
| **Apply with a chosen resume** | Browse Jobs | One click. The match score appears within seconds. A message explains if a rule blocks the application. |
| **Follow an application** | Dashboard, Browse Jobs, My Activity | Status badge, match ring, interview date, time, mode and venue. My Activity lists every change with the time. |
| **Waiting for a reply** | Dashboard | Shows how many days are left of the reply the recruiter promised. |
| **Edit profile** | Profile | Name, phone (10 digits) and address. |
| **Change password** | Profile | Send a code to the email, then enter the code and the new password twice. Works for Google accounts too. |
| **Delete account** | Profile | Code by email first. Everything of the candidate is removed. |
| **Emails** | Inbox | Welcome, resume uploaded, application submitted, shortlisted, interview scheduled or changed, rejected, hired, password changed, account deleted. |

**Not in the screenshot set:** the My Activity page (a timeline of every change to your applications, newest first).

# 13 Working guide: the recruiter

Follow the screens in the order a recruiter meets them: Dashboard, jobs, analytics, activity, profile.

## Recruiter: Dashboard (top)

The recruiter's home. The same layout as the candidate Dashboard, with hiring numbers.




**1 Profile and log out**
The name opens the Profile page. Log out ends the session.

**2 Recruiter menu**
Dashboard, My Jobs, Analytics and Activity Feed.

**3 Quick start**
Three steps for a new recruiter: post a job, review applicants, shortlist and schedule. Got it, hide this closes it.

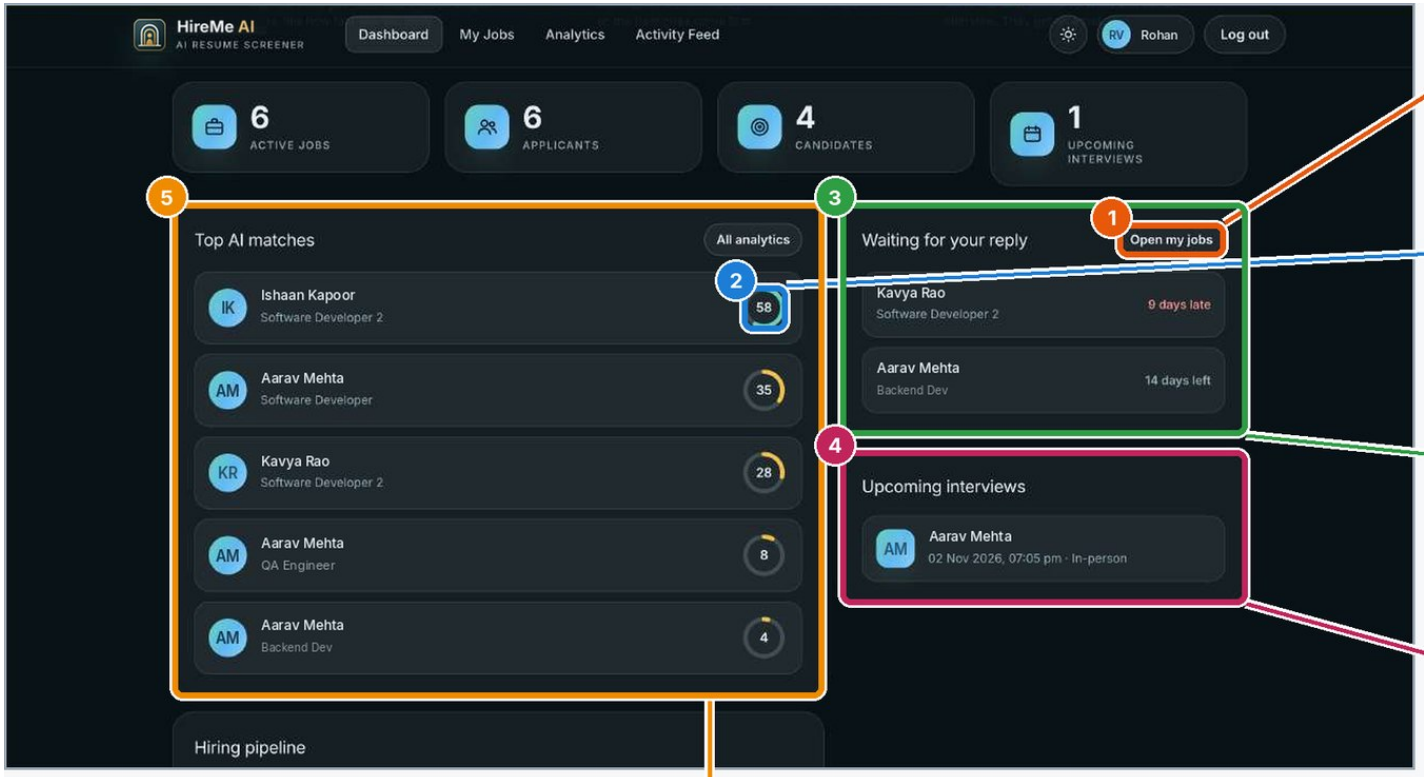
**4 Number cards**
Active jobs, Applicants, Candidates, Upcoming interviews. Point to read the meaning, click to open the page.

**5 Post a job**
Opens My Jobs with the post-a-job form.

**6 View analytics**
Opens the Analytics page: charts of applicants and outcomes.

# Recruiter: Dashboard (lower part)

Who to look at first, and who is still waiting for an answer.




**1 Open my jobs**
Jumps to My Jobs so the applicants can be answered.

**2 Match ring**
One number that sums up skill match and job fit.

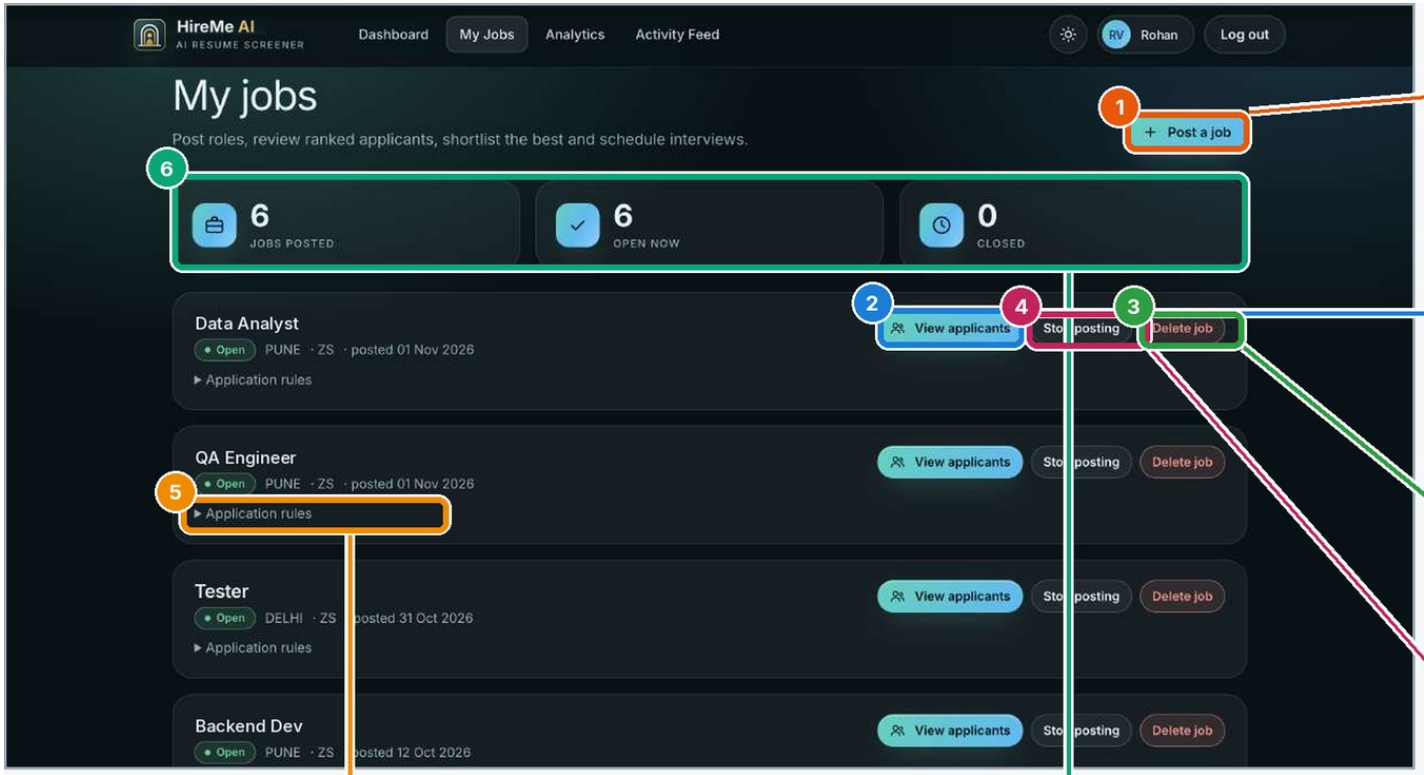
**3 Waiting for your reply**
Applicants who have not had an answer yet, the most late first. It uses the reply time the recruiter promised in the job rules, for example 9 days late in red or 14 days left.

**4 Upcoming interviews**
The next scheduled interviews with date, time and mode.

**5 Top AI matches**
The best-scoring applicants across all jobs. The ring on the right is the match score from 0 to 100: green is strong, amber is medium.

# Recruiter: My jobs

All the jobs of this recruiter, with the actions for each one.




**1 Post a job**
Opens the form for a new job: title, description, location, the skills that matter and the application rules (below).

**2 View applicants**
Opens the applicants of this job, ranked by match score. From there the recruiter can read the profile, shortlist, reject or schedule an interview.

**3 Delete job**
Removes the job together with its applications and interviews. The page asks to confirm.

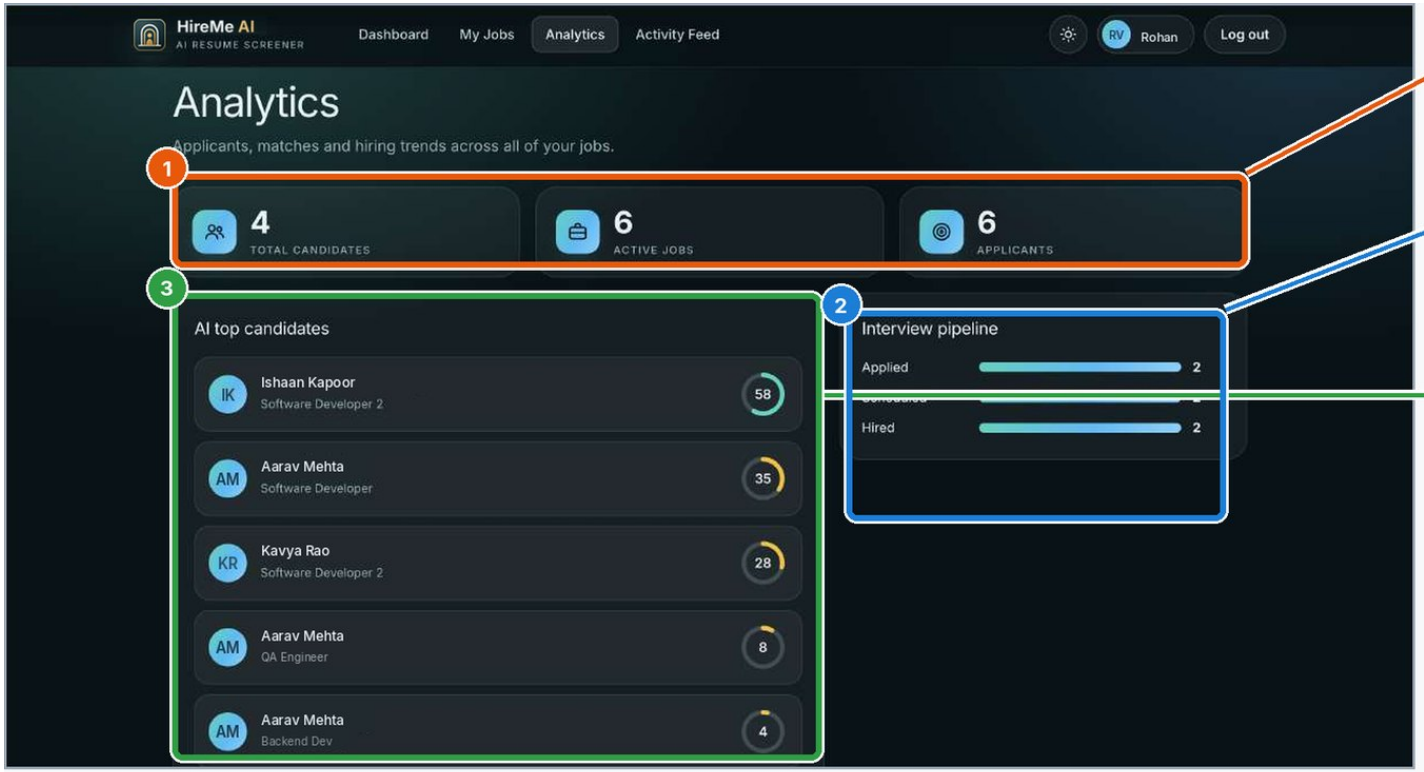
**4 Stop posting**
Closes the job so nobody can apply. It can be reopened later.

**5 Application rules**
Click to see the rules chosen for this job: one job at a time, a pause after hiring, a wait after rejecting and the promised reply time.

**6 Number cards**
Jobs posted, Open now, Closed. Jobs posted opens the form, the other two jump to the list.

# Recruiter: Analytics (summary)

Numbers for all jobs together.



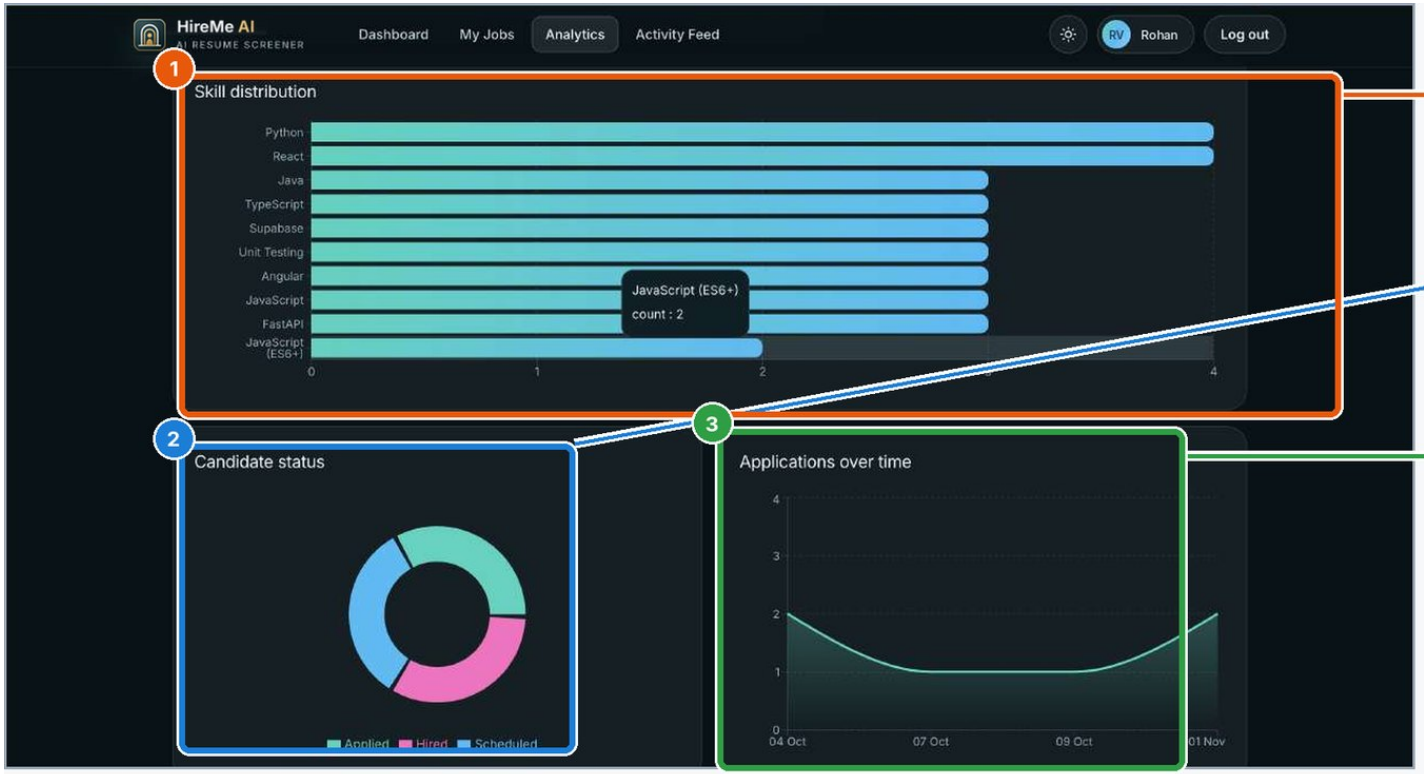

**1 Number cards**
Total candidates, Active jobs and Applicants.

**2 Interview pipeline**
How many applications sit in each stage (Applied, Scheduled, Hired).

**3 AI top candidates**
The highest match scores, with the job each person applied for.

# Recruiter: Analytics (charts)

Three charts built from the live data.



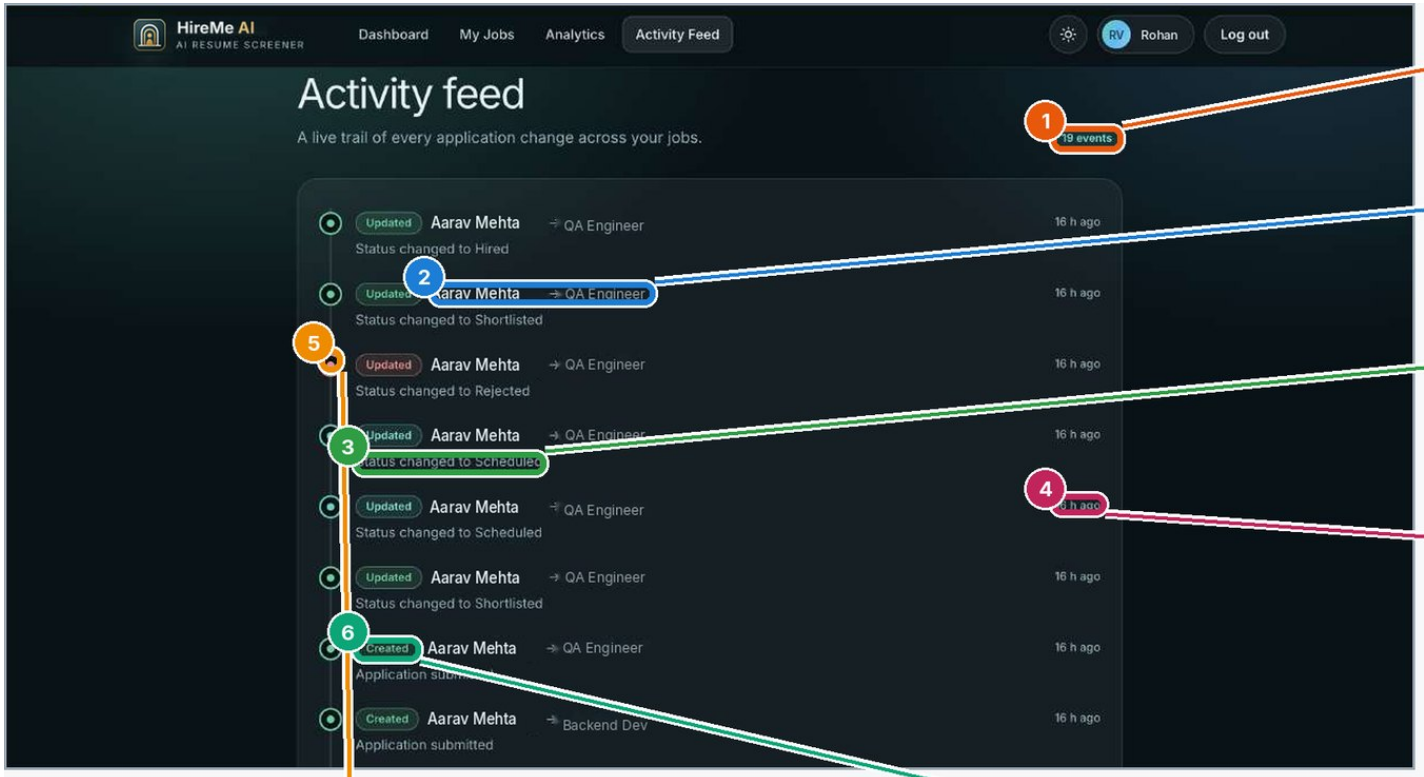

**1 Skill distribution**
Which skills appear most among the candidates. Point at a bar to see the exact count.

**2 Candidate status**
A ring chart of applicants by stage: Applied, Scheduled, Hired.

**3 Applications over time**
A line chart that shows on which days applications arrived.

# Recruiter: Activity feed

A permanent trail of every change on every application.




**1 Event count**
How many events are in the feed (19 here).

**2 Who and which job**
The candidate and the job the event belongs to.

**3 What happened**
The change in plain words, for example Status changed to Scheduled.

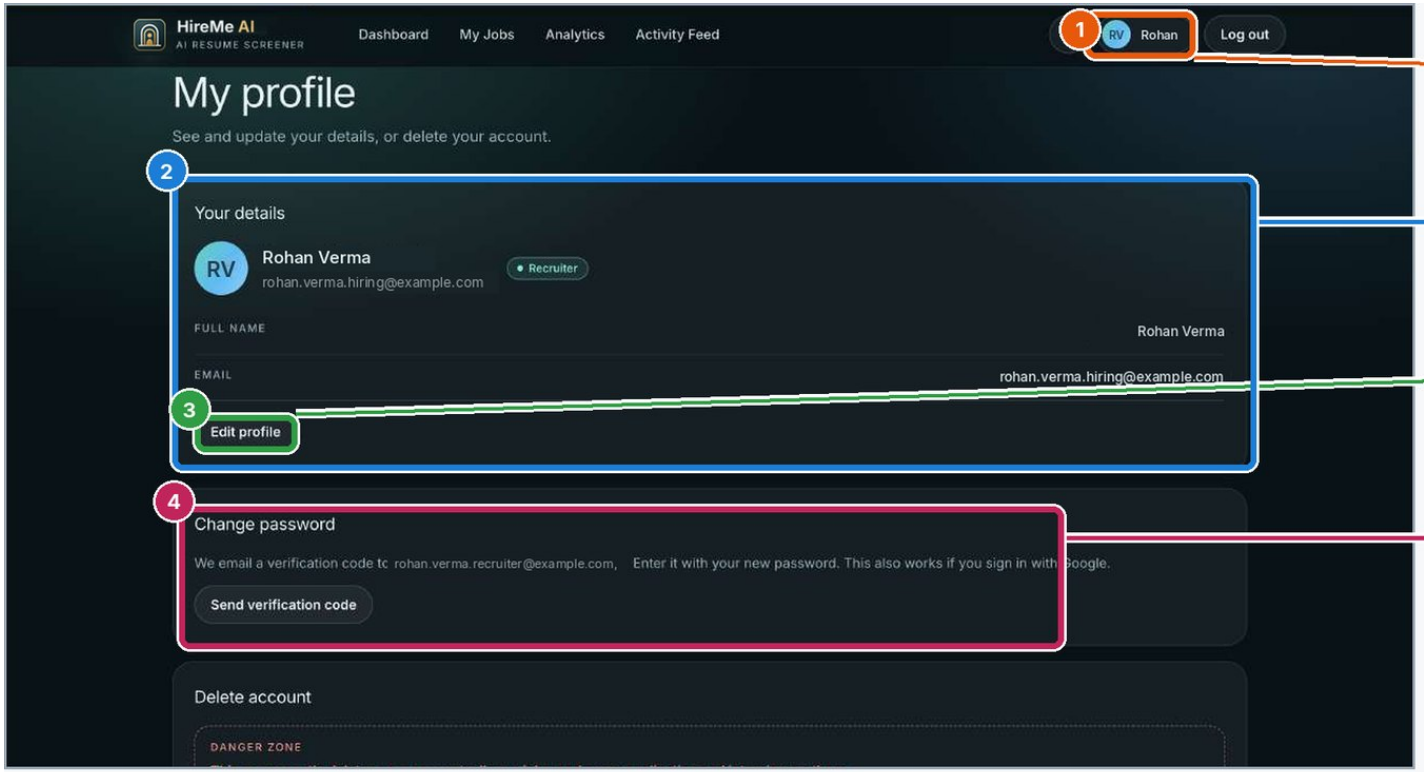
**4 When**
How long ago it happened. Nothing is lost: the feed is the audit trail.

**5 Red or green dot**
A green dot marks a normal step. A red dot marks a rejection, so it is easy to spot.

**6 Event type**
Created = a new application came in. Updated = a status changed.

# Recruiter: Profile page

The same page as for candidates, without phone and address.




**1 Profile button**
Opens this page from anywhere.

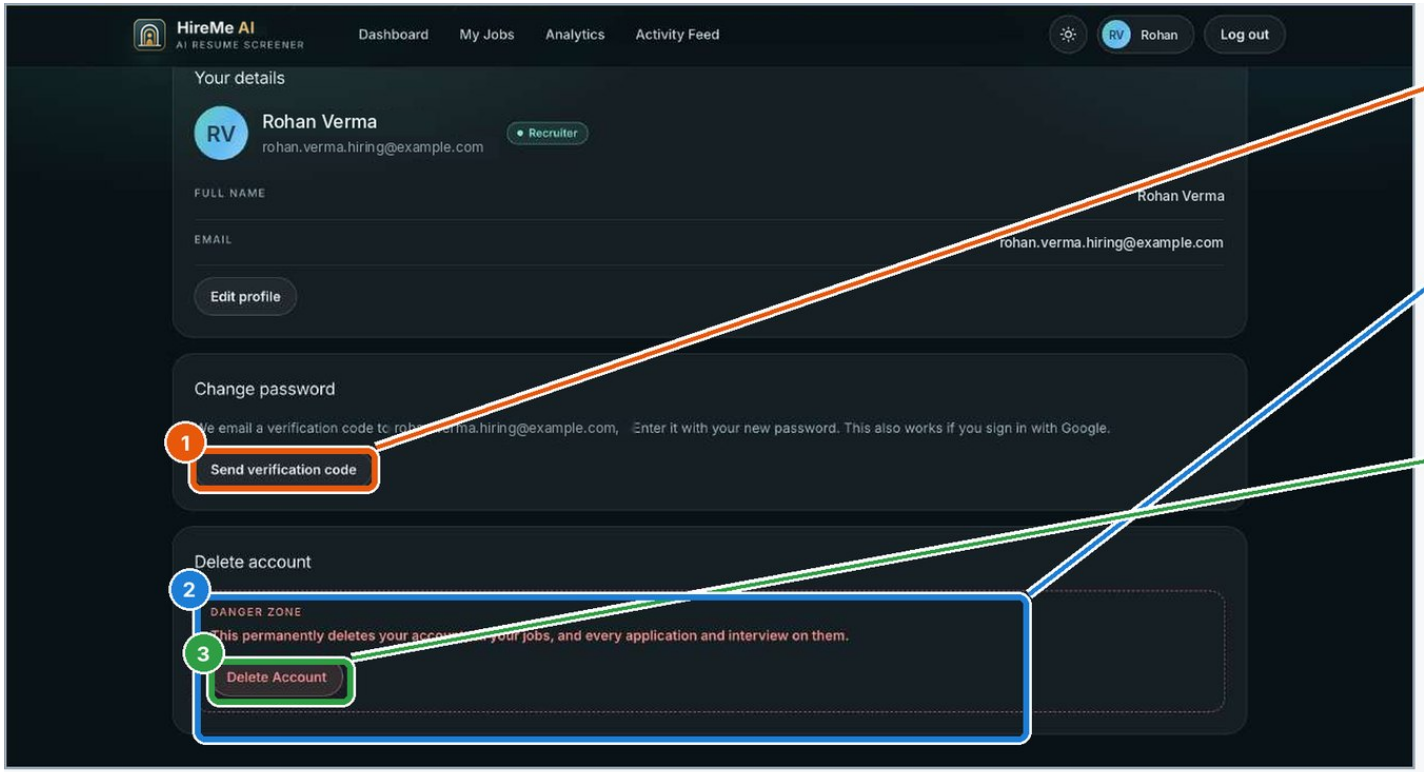
**2 Your details**
Name, email and the Recruiter badge. Recruiters can edit their name only.

**3 Edit profile**
Change the name shown in the top bar.

**4 Change password**
A code is sent to the email first. Then the new password is saved.

# Recruiter: delete account

For recruiters the warning is stronger, because jobs and applications go too.




**1 Send verification code**
Same flow as the candidate: a 6-digit code by email, then the new password.

**2 Danger zone**
Deleting a recruiter account permanently removes the account, all its jobs and every application and interview on them.

**3 Delete Account**
Needs the email code, so nobody can delete an account by accident.

## 13.1 Posting a job and choosing the rules

The post-a-job form opens from the **Post a job** button on My Jobs. It is not in the screenshot set, so here are its fields:

| Field | What to enter |
|---|---|
| **Title, description, location** | What the job is. The description is also what the AI compares resumes with. |
| **Skills** | The skills that matter. The AI can also read them from the description. |
| **Application rules** | The four rules from section 7.1: one job at a time, pause after a hire, wait after a rejection, and the reply promise in days. |
| **Recruiter terms tick** | A recruiter must accept short terms (use candidate data only for this hiring, follow the rules posted). The job cannot be posted without the tick. |

After posting, the recruiter gets a "Job is live" email with the rules written out. Candidates see the same rules on the job card.

## 13.2 Everything a recruiter can do

| Function | Where | How it works |
|---|---|---|
| **Create a recruiter account** | Create account | Same form as a candidate with the company name. Code by email, then a welcome email. |
| **Quick start** | Dashboard | Three steps for a new recruiter. Can be hidden and brought back. |
| **Number cards** | Dashboard, My Jobs, Analytics | Active jobs, Applicants, Candidates, Upcoming interviews. Hover to read, click to open. |
| **Post a job with rules** | My Jobs | See 13.1. |
| **Stop posting, delete a job** | My Jobs | Stop posting closes the job so nobody can apply. Delete job removes it with its applications and interviews, after a confirmation. |
| **View applicants** | My Jobs | Applicants ranked by match score with the matched skills; open the candidate profile and resume. |
| **Shortlist, reject, hire** | View applicants | Moves the application to the next stage. The candidate gets an email and the Activity feed records it. |
| **Schedule or change an interview** | View applicants | Date, time, mode (online or in person) and venue. The candidate gets an email with the time in IST. |
| **Waiting for your reply** | Dashboard | Applicants who have not had an answer, most late first, based on the reply promise. |
| **Top AI matches** | Dashboard, Analytics | The best scores across all jobs. |
| **Analytics** | Analytics | Skill distribution, candidate status ring chart, applications over time, interview pipeline. |
| **Activity feed** | Activity Feed | Every application change across all jobs, with who, what and when. |
| **Edit profile, change password, delete account** | Profile | Name only; password with a code by email; deleting removes the jobs and everything on them. |
| **Emails** | Inbox | Welcome, job is live, new applicant (with the score), password changed, account deleted, codes. |

# 14 API reference

All addresses start with the API address. "Login" means a valid token is needed. Request and response bodies are JSON (resume upload uses a file form).

| Area | Method and path | Login | What it does |
|---|---|---|---|
| **Auth** | POST /api/Auth/register/candidate and /register/recruiter | No | Creates an inactive account and emails a code (needs the consent tick and the human check) |
| **Auth** | POST /api/Auth/verify-otp and /resend-otp | No | Checks the code and activates the account; asks for a new code |
| **Auth** | POST /api/Auth/login | No | Email + password (+ human check). Returns the JWT |
| **Auth** | POST /api/Auth/google-login and /google-register | No | Login or sign-up with a Google ID token |
| **Auth** | POST /api/Auth/forgot-password and /reset-password | No | Sends a code; sets a new password with the code |
| **Roles** | GET /api/Roles/register-options | No | Roles that can be chosen at sign-up |
| **Users** | GET and PUT /api/Users/profile/{id} | Yes | Read or update your own profile |
| **Users** | POST /api/Users/me/request-delete-otp and DELETE /api/Users/me | Yes | Code for deleting, then delete your own account |
| **Password** | POST /api/Users/me/password-code and PUT /api/Users/me/password | Yes | Send a code; change the password with the code |
| **Resumes** | POST /api/Resumes/upload/{userId} | Yes | Upload a PDF or DOCX; the AI reads it |
| **Resumes** | GET /api/Resumes/candidate/{userId}, GET .../download/{resumeId}, DELETE .../{resumeId} | Yes | List, download, delete resumes |
| **Jobs** | POST /api/Jobs; GET /api/Jobs; GET /api/Jobs/my-jobs; GET, PUT, DELETE /api/Jobs/{id} | Yes | Post, list open jobs, list own jobs, read, edit (open or close), delete |
| **Applications** | POST /api/Applications | Yes | Apply (rules are checked, score is made) |
| **Applications** | GET /api/Applications/my-applications and /job/{jobId} | Yes | Candidate: own applications. Recruiter: applicants of a job |
| **Applications** | GET /api/Applications/{id}/candidate-profile, PUT .../{id}/status, GET .../{id}/activity-log | Yes | Profile of an applicant, change the stage, history of one application |
| **Applications** | GET /api/Applications/activity-feed and /my-activity-feed | Yes | Recruiter feed; candidate feed |
| **Interviews** | POST /api/Interviews; PUT /api/Interviews/{id}; GET /api/Interviews; GET /api/Interviews/my-interviews | Yes | Schedule, change, list for a recruiter, list for a candidate |
| **Dashboard** | GET /api/Dashboard/overview | Yes | Numbers, top matches and pipeline for the recruiter Dashboard |
| **AiMatching** | GET /api/AiMatching/match-score/{id}, /dashboard-report, /global-matches | Yes | Score details and reports built from the stored scores |

The consent endpoint (records that the visitor accepted the first-visit notice) and the human-check verification are also part of the API but are not listed with their paths here; see the Controllers folder.

# 15 Emails

Every email uses one template (EmailTemplate.cs): a brand header (HireMe AI, AI Resume Screener), the message, a button (for example Go to my dashboard), and a footer with the company name and links to the Terms and the Privacy Policy. A plain-text version is always sent as well, so an email still reads well in any mail program.

| Email | Sent when | To |
|---|---|---|
| **Verification code** | Sign-up, resend code, forgot password, change password, delete account | The user |
| **Welcome** | The first time an account becomes active (candidate and recruiter versions; also after Google sign-up) | The user |
| **Resume uploaded** | A resume is saved and analysed | Candidate |
| **Application submitted** | After Apply; mentions the reply the recruiter promised | Candidate |
| **New applicant** | After Apply; says who applied and the match score | Recruiter |
| **Shortlisted** | The recruiter shortlists the application | Candidate |
| **Interview scheduled or changed** | The recruiter sets or edits an interview; date and time in IST, mode, venue | Candidate |
| **Rejected** | The recruiter rejects; mentions the wait before applying again | Candidate |
| **Hired** | The recruiter hires; mentions the pause on other jobs | Candidate |
| **Job is live** | A job is posted; lists the rules | Recruiter |
| **Password changed** | After a reset or a change on the Profile page | The user |
| **Account deleted** | After an account is deleted | The user |

A failed email never undoes the action: if the mail server is down, the application is still saved and the error is only logged.

# 16 Testing, deployment and settings

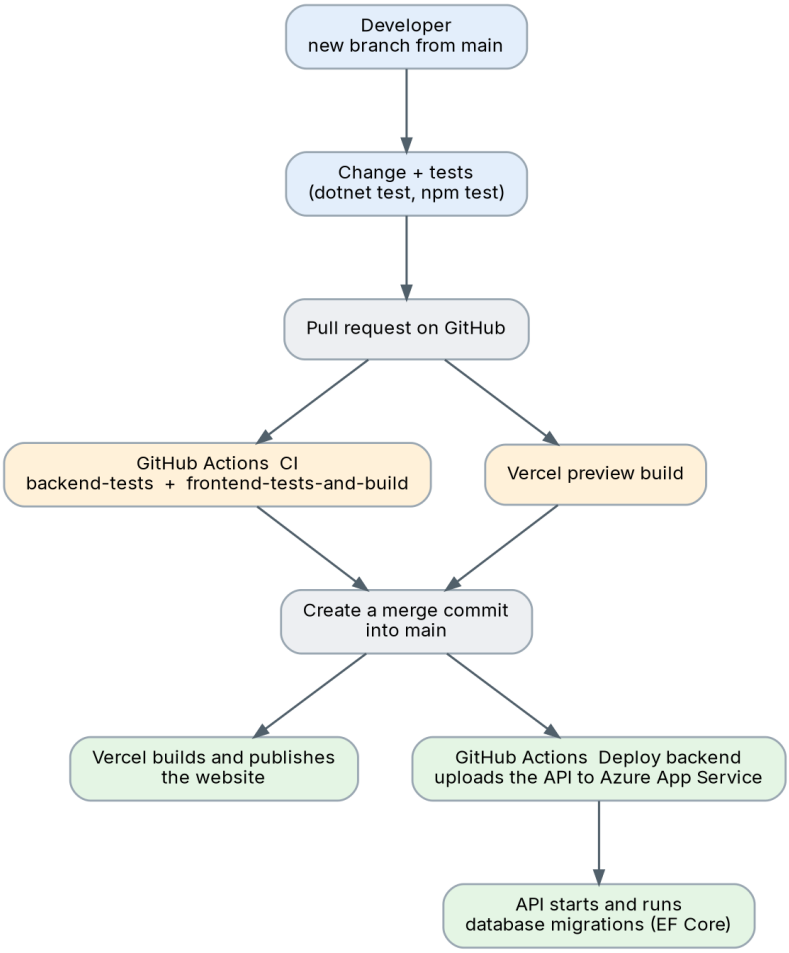



Figure 11. From a change to the live site.

## 16.1 Automatic tests

- **Backend:** 121 xUnit tests (run with **dotnet test BackendAPI.Tests**).
- **Frontend:** 131 Vitest tests in 28 files (run with **npm test** inside the Frontend folder), plus **npm run build**.
- GitHub Actions runs both on every push and pull request. A pull request is merged only when the checks are green.
- Tests also guard the content: the contact email, the legal pages without placeholders, a single font, and the agreed words (Log in, Log out, Create account).

## 16.2 Settings the system needs (names only, no values)

| Where | Setting | Used for |
|---|---|---|
| **Azure App Service** | Database connection string; JWT key, issuer and lifetime; GoogleAuth:ClientId; SMTP (Brevo) login; AI service address and key | Database, login tokens, Google check, emails, calls to the AI service |
| **Azure App Service** | Turnstile__SecretKey | Server-side check of the human test (remove it to switch the check off in an emergency) |
| **Azure App Service** | EmailBranding__Address, EmailBranding__SupportEmail (optional) | Extra lines in the email footer |
| **Vercel** | VITE_TURNSTILE_SITE_KEY, API address | The human test widget and the address of the API |
| **AI service** | GROQ_API_KEY, GROQ_MODEL, AI_SERVICE_KEY | Language model access and the shared key with the API |
| **GitHub** | AZURE_WEBAPP_PUBLISH_PROFILE (secret) | Lets the Deploy backend workflow publish to Azure |

## 16.3 Running it on a laptop

- Start the database: **open -a Docker; docker start sqlserver**
- API: **cd BackendAPI && dotnet run** (listens on http://localhost:5140)
- Website: **cd Frontend && npm run dev** (http://localhost:5173)
- Local and live use different users and jobs, so test one side at a time.

# 17 Glossary

| Term | Meaning |
|---|---|
| **API** | The server program the website talks to. It holds the rules and the database access. |
| **JWT** | A signed login token. It says who you are and for how long, and is sent with every request. |
| **OTP** | One-time password: a short code valid for a few minutes, sent by email. |
| **Entity Framework Core (EF Core)** | The library that turns C# classes into database tables and queries. |
| **Migration** | A saved step that changes the database structure the same way everywhere. |
| **Repository** | A class that hides the database queries for one table. |
| **CORS** | A browser rule. The API says which website addresses may call it. |
| **Embedding** | A list of numbers that stands for the meaning of a word or text. |
| **Cosine similarity** | A number from 0 to 1 that says how close the meaning of two embeddings is. |
| **Turnstile** | Cloudflare's human check, a friendlier replacement for puzzles. |
| **Consent log** | The record that a person accepted the Terms and the Privacy Policy, with version and time. |
| **CI/CD** | Automatic testing (CI) and automatic publishing (CD) after each merge. |
| **Pull request** | A proposed change on GitHub that is tested and reviewed before it joins main. |
| **Single-page app** | A website that loads once and then changes pages without a full reload. |